Program 1:

```
#include <iostream>
#include <stack>
#include <string>
#include <deque>
using namespace std;


string reverseString(string& text)
{
   // std::stack<char> st;
     std::deque<char> st;
    for (char c : text)
          //st.push(c);
               st.push_back(c);

    string result;
    while (!st.empty())
    {
       // result += st.top();
       // st.pop();
          result += st.back();
          st.pop_back();
    }
    return result;
}
int main()
{
        string text;
         cout << "Enter text: ";
         //getline(cin, text);
           cin>> text;
         cout << "Reversed = " << reverseString(text) << "\n";
    system("pause");
    return 0;
}
```

Using stack

Using deque

Using stack

Using deque

Recursion& search

The Basic Idea

- *Recursion* is an extremely powerful problem-solving technique.

- **Recursion** tries to
  - break a problem into ***smaller identical problems***.
  - solve the problem by solving **smaller** instances of the **same** problem.

    (The solutions of the smaller problems will lead to the solution of the original problem.)

- Problems that at first appear to be quite difficult often have simple recursive solutions.

About Efficiency

- The technique of recursion, though powerful, does not guarantee you an efficient solution.

- Some recursive solutions are so **inefficient** that they should not be used.
  - Overhead associated with function calls.
  - Inherent inefficiency of some recursive algorithms.

An Example (1/2)

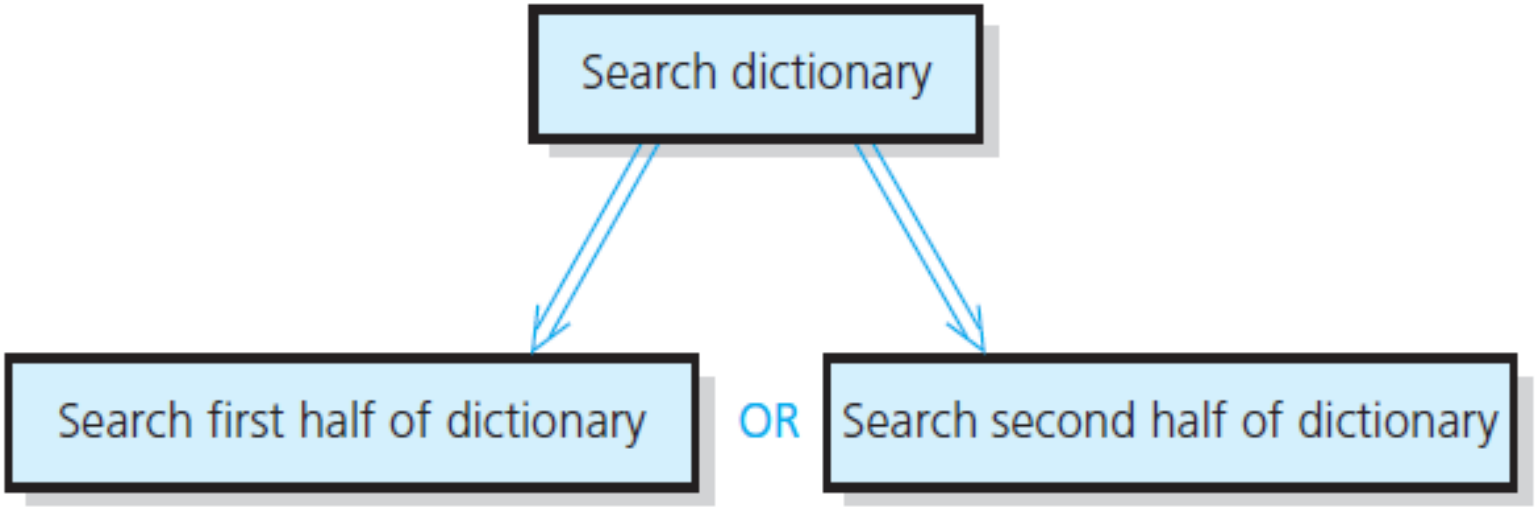
- Looking up a word in a dictionary using *binary search*:

```
if (the dictionary contains only one page)
   Scan the page for the word
else {
   Open the dictionary to a point near the middle
   Determine which half contains the word

   if (the word is in the first half)
      Search the first half for the word
   else
      Search the second half for the word
}
```

An Example (2/2)

- The *binary search* strategy reduces the search problem to a smaller instance of the same problem:



Deriving a Recursive Solution (1/2)

- Binary search reduces the problem of searching the dictionary for a word to a problem of searching half of the dictionary for the word.

- Two important points:
  - Once you have divided the dictionary in halves, you already know how to search the appropriate half.
  - There is a special case that is different from all the other cases: dictionary with a single page.
    - **Stop dividing** and **solve the problem directly**.
    - This special case is called the **base case** (or basis).

Deriving a Recursive Solution (2/2)

- Binary search falls into the general strategy of **divide and conquer**.
  - You solve the dictionary search problem by first *dividing* the dictionary into two halves.
  - And then *conquering* the appropriate half (a smaller problem).
  - You solve the smaller problem by using the *same* divide-and-conquer strategy until you reach the base case.
- Many recursive solutions utilize the divide-and-conquer strategy.

The Search Example Revisited

- More rigorous pseudocode:

```
search(aDcitionary: Dictionary, word: string)

   if (aDictionary is one page in size)
      Scan the page for word
   else {
      Open aDictionary to a point near the middle
      Determine which half contains word

      if (word is in the first half of aDictionary)
         search(first half of aDictionary, word)
      else
         search(second half of aDictionary, word)
   }
```

Observations from a Recursive Solution

- A recursive function **calls itself**.
  - E.g., the `search` function calls `search(first/second half of aDictionary, word)`.
- Each recursive call solves an **identical**, but **smaller**, problem.
- The solution to **at least one** smaller problem—the **base case**—is known.
  - When you reach the base case, the recursive calls stop and you solve the problem directly.
  - E.g., the function `search` scan the page for the word when the dictionary contains only one page.
- The diminishing size of the problem ensures that you will **eventually reach the base case**.

Binary Search Algorithm

- Searching a list for a particular value
  - *sequential* and *binary* are two common algorithms
- *Sequential search* (aka *linear search*):
  - Not very efficient
  - Easy to understand and program
- *Binary search*:
  - more efficient than sequential
  - but *the list must be sorted first!*

Why Is It Called "Binary" Search?

Compare sequential and binary search algorithms:

How many elements are eliminated from the list each time a value is read from the list and it is not the "target" value?

Sequential search: *only one item*

Binary search: *half the list*!

That is why it is called *binary* -

each unsuccessful test for the target value

reduces the remaining search list by 1/2.

Binary Search Method

- `find(target)`
- calls `search(array, target, first, last)`
- returns the index of the entry if the target value is found or -1 if it is not found
- Compare it to the pseudocode for the "name in the phone book" problem

```
int search(int [],int target, int first, int last)
 {
    int location = -1; // not found

    if (first <= last) // range is not empty
    {
       int mid = (first + last)/2;

       if (target == a[mid])
          location = mid;
       else if (target < a[mid]) // first half
          location = search(target, first, mid - 1);
       else //(target > a[mid]) second half
          location = search(target, mid + 1, last);
    }

    return location;
 }
```

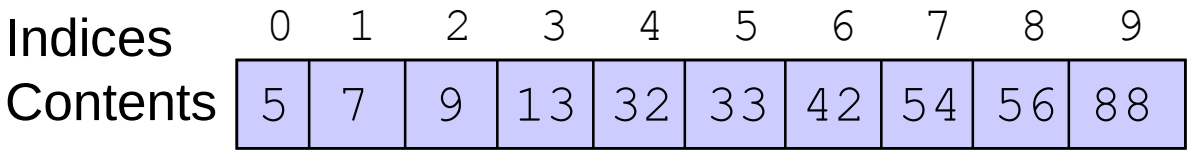
Where is the composition?

- If no items
  - not found (-1)
- Else if target is in the middle
  - middle location
- Else
  - location found by search(first half) or search(second half)

Binary Search Example

`target` is `33`

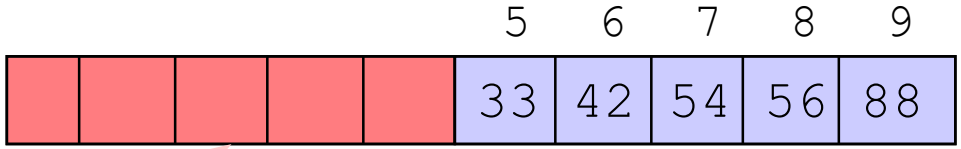
The array **`a`** looks like this:



`mid = (0 + 9) / 2` (which is `4`)

`33 > a[mid]` (that is, `33 > a[4]`)

So, if `33` is in the array, then `33` is one of:

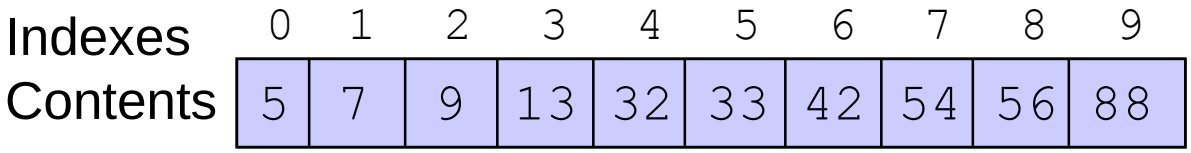


Eliminated half of the remaining elements from consideration because array elements are sorted.

Binary Search Example

`target` is `33`

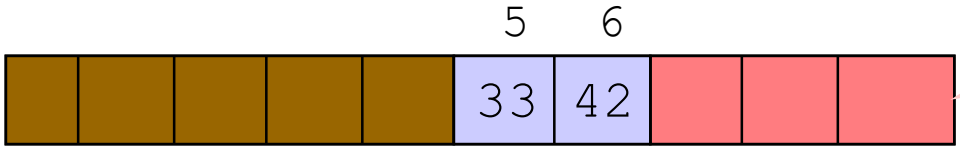
The array **`a`** looks like this:



`mid = (5 + 9) / 2` (which is `7`)

`33 < a[mid]` (that is, `33 < a[7]`)

So, if `33` is in the array, then `33` is one of:

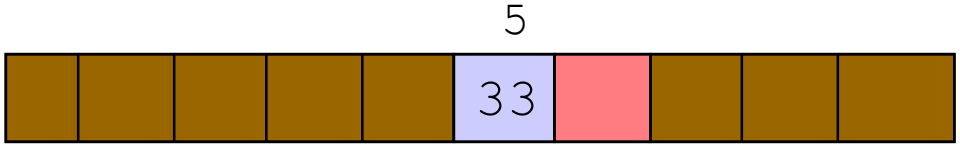


Eliminate half of the remaining elements

`mid = (5 + 6) / 2` (which is `5`)

`33 == a[mid]`

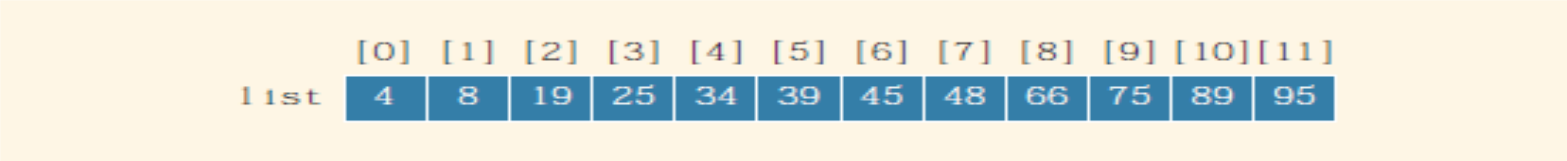
So we found `33` at index `5`:

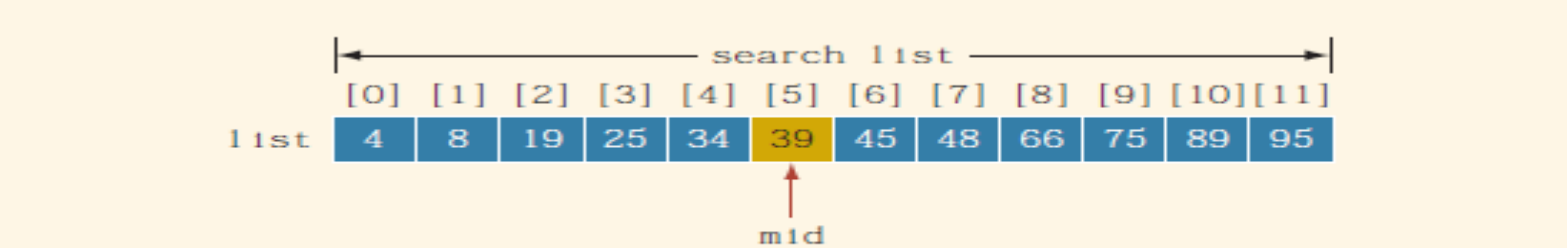


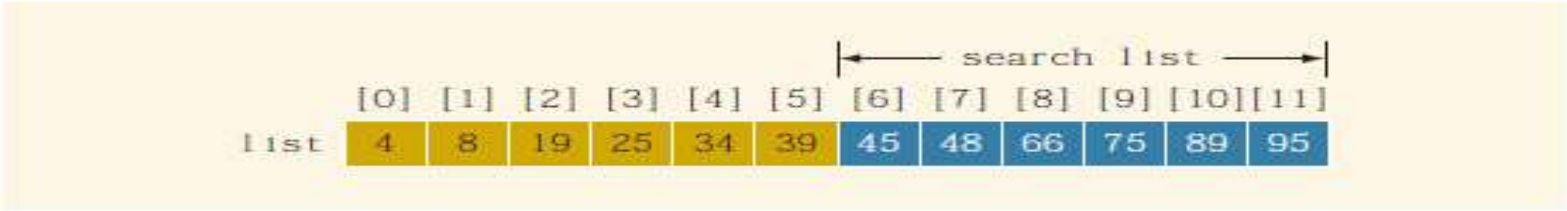
Sequential Search

- The sequential search (also called a linear search).
- The sequential search works the same for both array-based and linked lists. The search.
- always starts at the first element in the list and continues until either the item is found in the list or the entire list is searched.

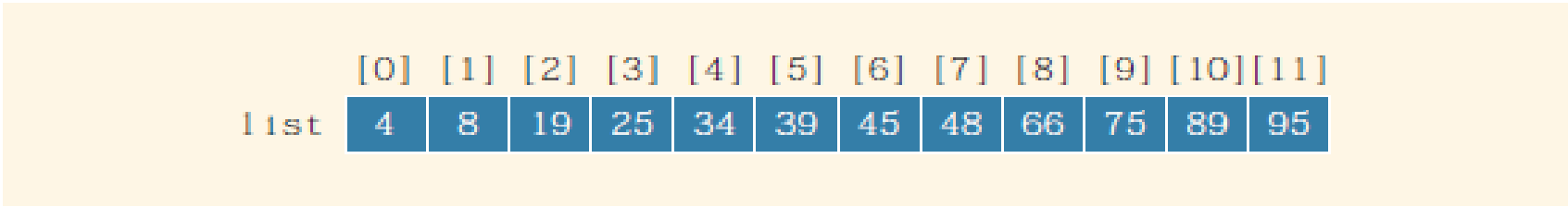
Binary Search







EXAMPLE :Suppose that we are searching for item 89.



Iteration	first	last	mid	list[mid]
1	0	11	5	39
2	6	11	8	66
3	9	11	10	89

Four Questions to Ask/Answer

- How can you define the problem in terms of smaller problems of the same type?
- How does each recursive call diminish the size of the problem?
- What instance(s) of the problem can serve as the base case?
- As the problem size diminishes, will you reach this base case?

A Recursive Function: Factorial (1/4)

- An **iterative definition** of *factorial*(*n*):

  factorial(*n*) = *n* * (*n* – 1) * (*n* – 2) * … * 1

  for any integer *n* > 0

  factorial(0) = 1

  (Note: the factorial of a negative integer is undefined.)

- A **recursive definition** of *factorial*(*n*)

  factorial(*n*) = *n* * *factorial*(*n*–1) if *n* > 0

  = 1 if *n* = 0

  (Note: the base case is deliberately given later.)

A Recursive Function: Factorial (2/4)

- It is easy to construct a C++ function that implements the definition.

```
/** Computes the factorial of the nonnegative integer n.
 @pre  n must be greater than or equal to 0.
 @post  None.
 @return  The factorial of n; n is unchanged. */
int fact(int n)
{
   if (n == 0)
      return 1;
   else // n > 0
      return n * fact(n-1);
} // end fact
```

base case

A Recursive Function: Factorial (3/4)

- Does function `fact` fit the mold of a recursive solution?
  - ✓ One action of `fact` is to call itself.
  - ✓ Each recursive call to `fact` diminishes the size of the problem by 1.
  - ✓ `fact(0)` is the base case.
  - ✓ Given that `n` is nonnegative, you always reach the base case.

- Note that `fact` assumes the input `n` is non-negative, as stated in its precondition.

Function Call Stack (Recursion Simulation)

Every time a program calls a function, the computer creates a "Stack Frame" containing the function's variables and the return address, then **pushes** it onto the System Call Stack. When the function finishes, it is **popped**.

Call Stack

- When you call a function, the system sets aside space in memory for that function to do its necessary work.
  - We frequently call such chunks of memory **stack frames** or **function frames**.

- More than one function's stack frame may exist in memory at a given time.

Call Stack

- These frames are arranged in a **stack**. The frame for the most-recently called function is always on the top of the stack.

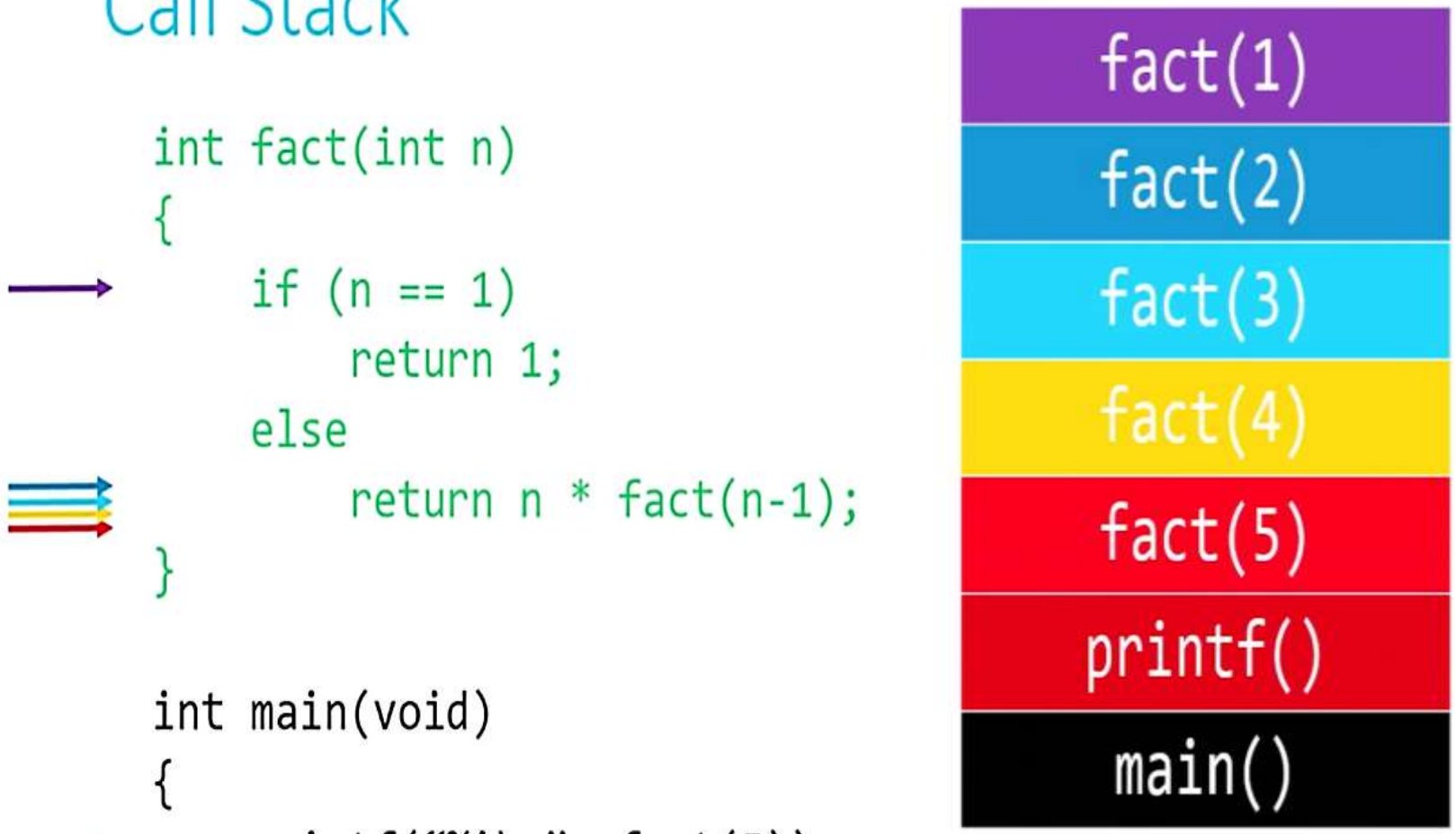
- When a new function is called, a new frame is **pushed** onto the top of the stack and becomes the active frame.

- When a function finishes its work, its frame is **popped** off of the stack, and the frame immediately below it becomes the new, active, function on the top of the stack. This function picks up immediately where it left off.

Call Stack

```
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n * fact(n-1);
}

int main(void)
{
    printf("%i\n", fact(5));
}
```

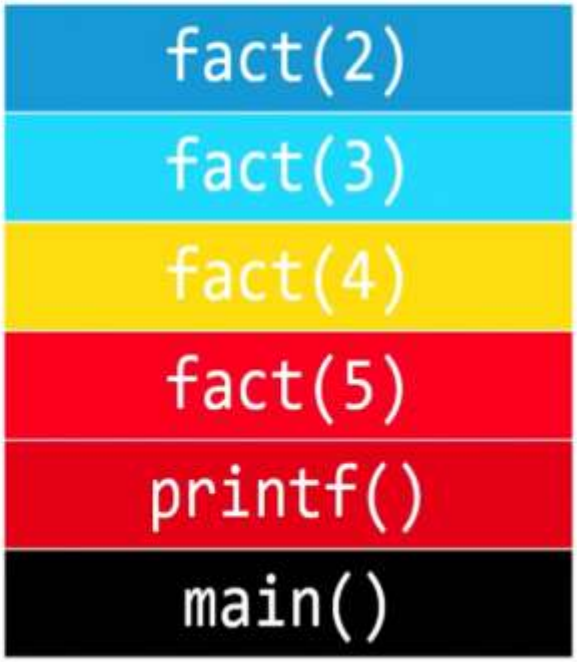


After evaluating fact(1), pop fact(1).

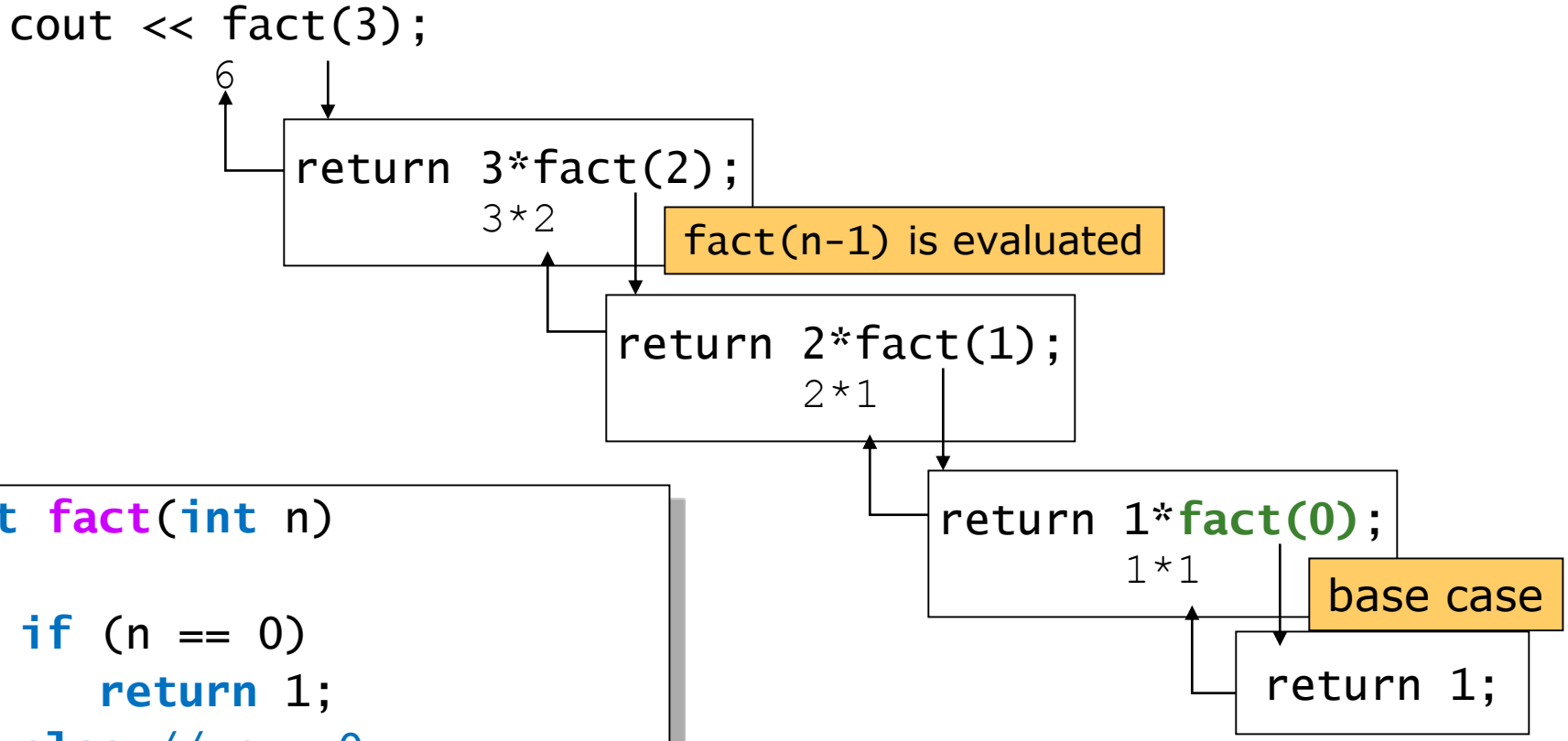
Call Stack

```
int fact(int n)
{
    if (n == 1)
        return 1;
    else
        return n * fact(n-1);
}

int main(void)
{
    printf("%i\n", fact(5));
}
```

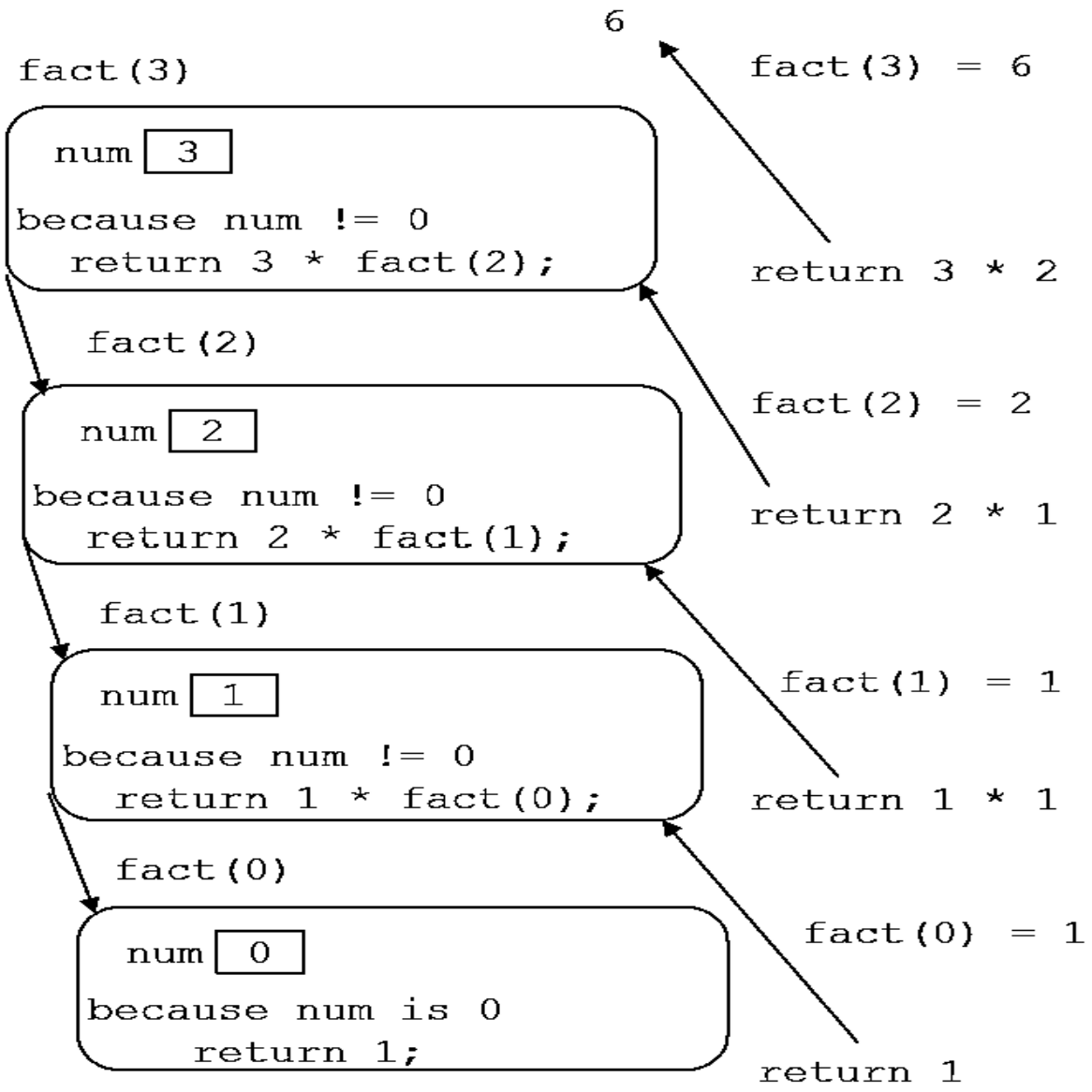


A Recursive Function: Factorial (4/4)




```
int fact(int n)
{
   if (n == 0)
      return 1;
   else // n > 0
     return n * fact(n-1);
} // end fact
```

Execution of fact(4)



From the previous example (factorial), it is clear that:

1. Every recursive definition must have one (or more) base cases.
2. The general case must eventually be reduced to a base case.
3. The base case stops the recursion.

To design a recursive function, you must do the following:

1. Understand the problem requirements.
2. Determine the limiting conditions. For example, for a list, the limiting condition is the number of elements in the list.
3. Identify the base cases and provide a direct solution to each base case.
4. Identify the general cases and provide a solution to each general case in terms of smaller versions of itself.

Box Traces

- A **box trace** is an illustration of how a compiler usually implements recursion.

- You may use it to help you understand recursion and to **debug** a recursive function.

- Each box in a box trace roughly corresponds to an "activation record."

Constructing a Box Trace (1/7)

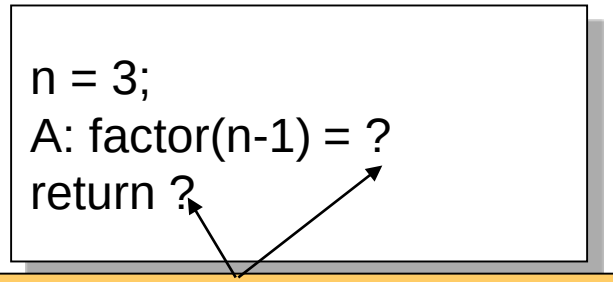
1. **Label each recursive call** in the body of the recursive function.
   - These labels help you keep track of the correct place to which you must return after a function call completes.

```
int fact(int n)
{  if (n == 0)
      return 1;
   else
      return n * fact(n-1);
}                      (A)
```

Constructing a Box Trace (2/7)

2. During the course of an execution, represent each call to the function by a new **box**, containing:
   - A copy of the actual value arguments.
   - A placeholder for the value returned by each recursive call from the current box.
     - Label this placeholder to correspond to the label in Step 1.
   - The value of the function itself.
   - The function's local variables.

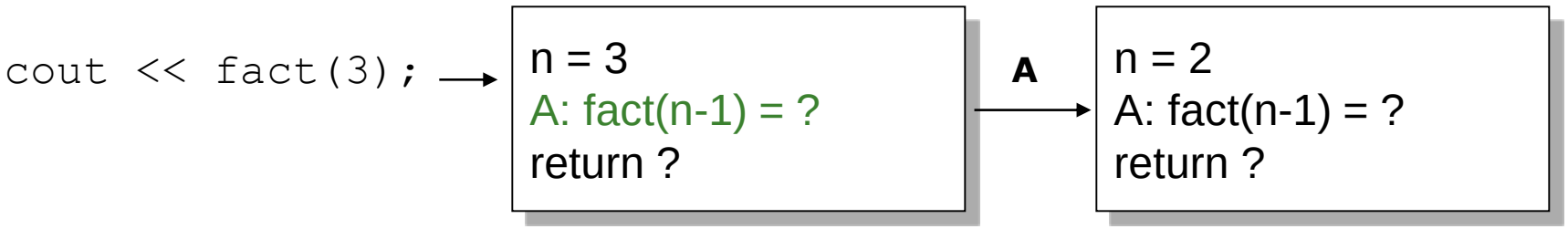
```
cout << fact(3);
```



when you first create a box, you will know only the values of the input arguments. You fill in the values of the other items as you determine them from the function's execution.

Constructing a Box Trace (3/7)

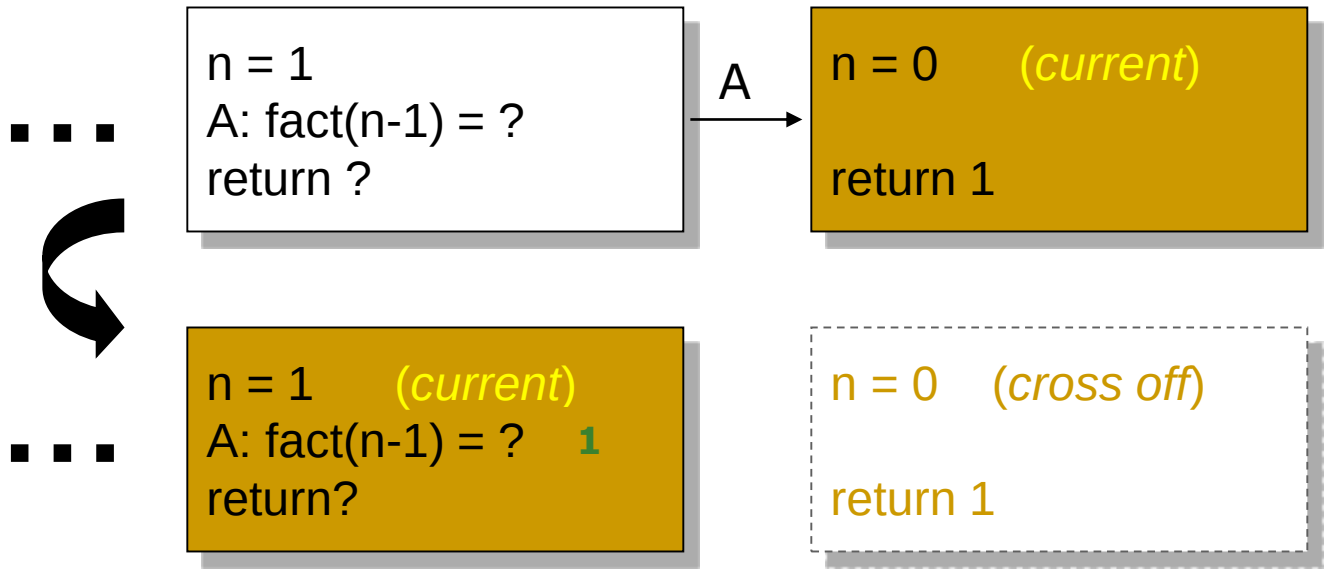
3. When you create a new box after a recursive call, draw an **arrow** from the box that makes the call to the newly created box.
   - Label each arrow to correspond to the label of the recursive call.



4. After you create the new box and arrow, start executing the body of the function.

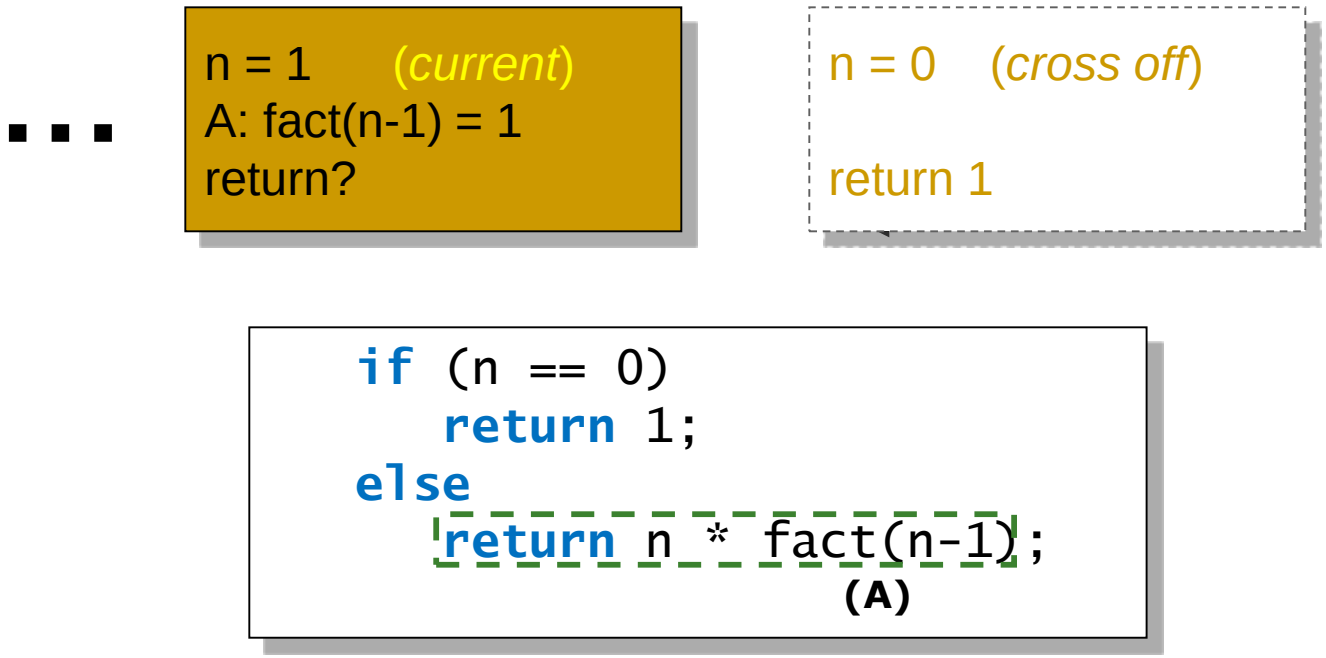
Constructing a Box Trace (4/7)

5. On **existing** the function, cross off the current box and follow its arrow back to the box that called the function.
   - **Substitute the value** returned by the just-terminated function call into the appropriate item in the current box.



Constructing a Box Trace (5/7)

- When you return to point A and substitute the computed value for `fact(n-1)`:
  - Continue execution by evaluating the expression `return n * fact(n-1).`

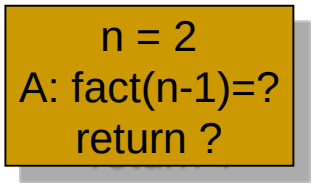


Constructing a Box Trace (6/7)

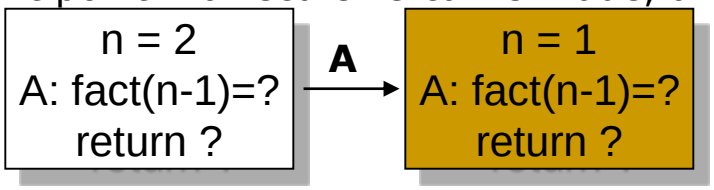
- Box trace of `fact(2)`:

```
int fact(int n)
{  if (n == 0)
      return 1;
   else
      return n * fact(n-1);
}                                    (A)
```

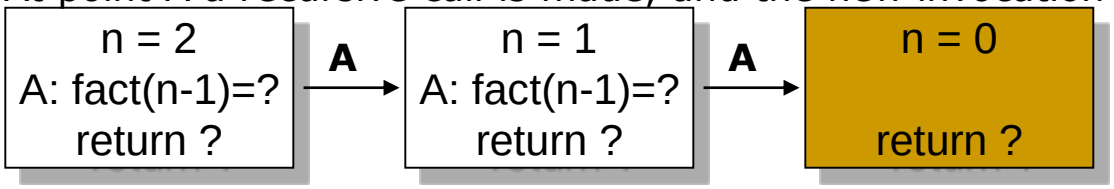
The initial call is made, and `fact` begins execution:



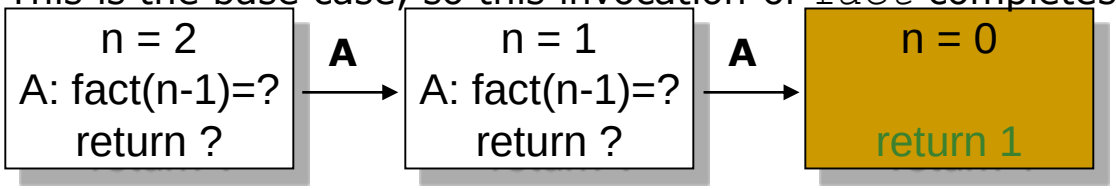
At point A a recursive call is made, and the new invocation of `fact` begins execution:



At point A a recursive call is made, and the new invocation of `fact` begins execution:

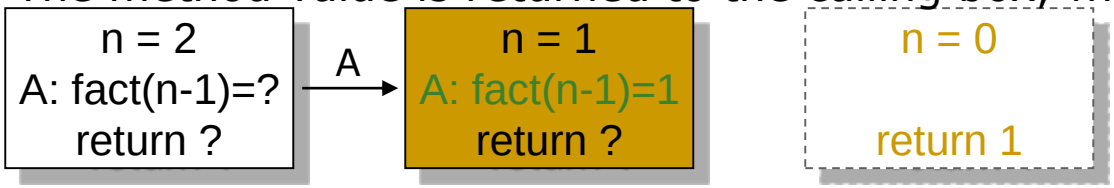


This is the base case, so this invocation of `fact` completes:



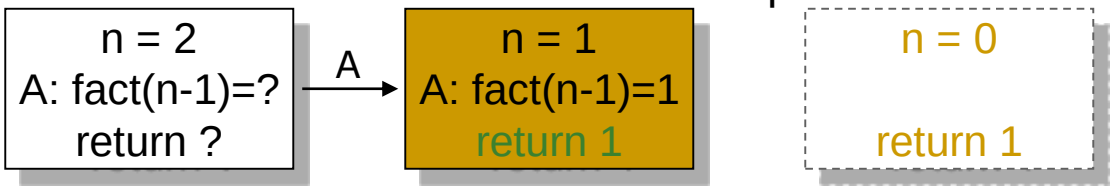
Constructing a Box Trace (7/7)

The method value is returned to the calling box, which continues execution:

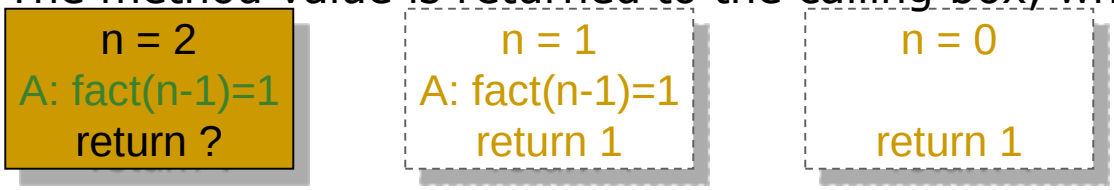



```
int fact(int n)
{  if (n == 0)
      return 1;
   else
      return n * fact(n-1);
}                          (A)
```

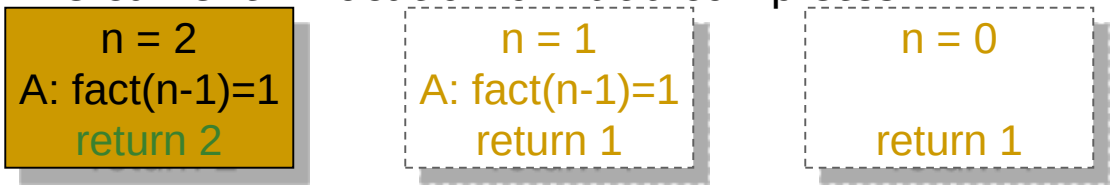
The current invocation of `fact` completes:



The method value is returned to the calling box, which continues execution:



The current invocation of `fact` completes:



The value `2` is returned to the initial call.

Invariants for Recursive Functions

- Writing ***invariants*** for recursive functions is as important as writing them for iterative functions.
  - An invariant is a condition that is **always true** at a particular point in an algorithm/program.

```
/**
 @pre  n must be greater than or equal to 0.
 @return  the factorial of n. */
int fact(int n)
{
   if (n == 0)
      return 1;
   else
      // Invariant: n > 0, so n-1 >= 0.
      // Thus, fact(n-1) returns (n-1)!
      return n * fact(n-1);
} // end fact
```

The recursive call satisfies `fact`'s precondition, so you can expect from the post-condition that `fact(n-1)` will return the factorial of `n-1`.

Return a Value

Recursion Example: Powers

- Recall predefined function pow():

  result = pow(2.0,3.0);

  - Returns 2 raised to power 3 (8.0)
  - Takes two double arguments
  - Returns double value
- Let's write recursively
  - For simple example

Function Definition for power()

```
int power(int x, int n)
{
        if (n<0)
        {
                cout << "Illegal argument";
                exit(1);
        }
        if (n>0)
                return (power(x, n-1)*x);
        else
                return (1);
}
```

Calling Function power()

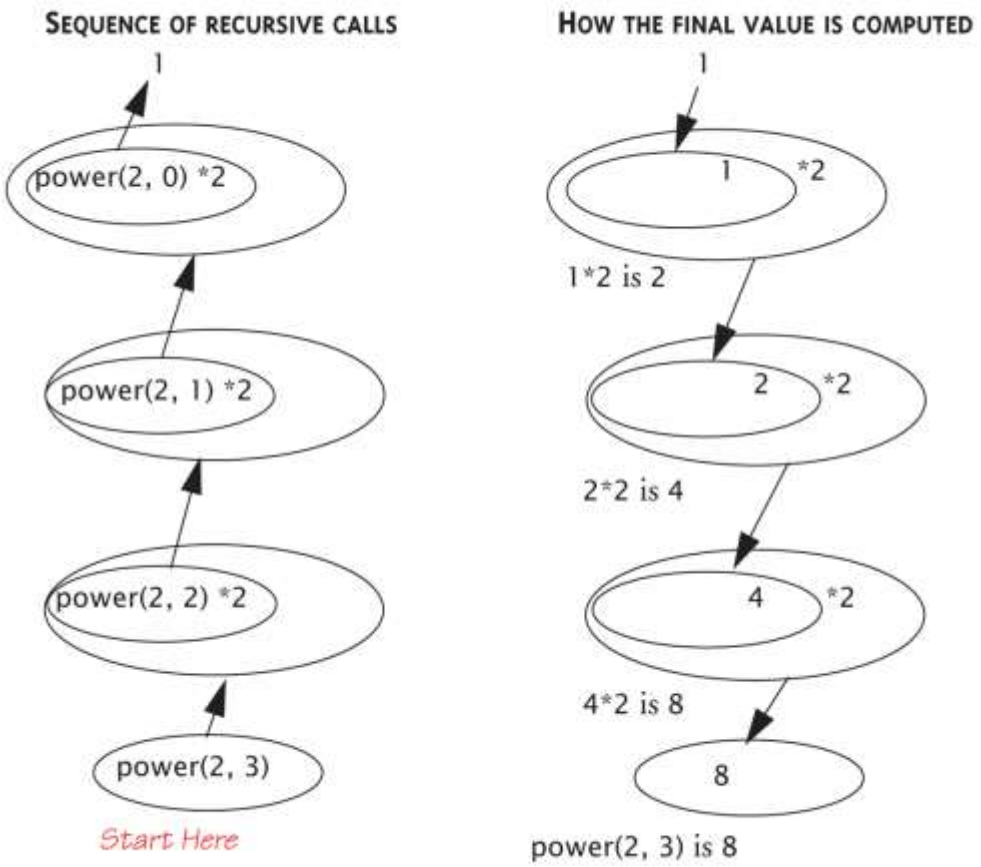
- Example calls:
- power(2, 0);
  → returns 1
- power(2, 1);
  → returns (power(2, 0) * 2);
  → returns 1
  - Value 1 multiplied by 2 & returned to original call

Calling Function power()

- Larger example:
  power(2,3);
  → power(2,2)*2
  → power(2,1)*2
  →power(2,0)*2
  →1
  - Reaches base case
  - Recursion stops
  - Values "returned back" up stack

Tracing Function power():
Evaluating the Recursive Function Call power(2,3)

Display 13.4 **Evaluating the Recursive Function Call power(2,3)**



Recursive void Function: Vertical Numbers

- Task: display digits of number vertically, one per line
- Example call:
  writeVertical(1234);
  Produces output:
  1
  2
  3
  4

Vertical Numbers: Recursive Definition

- Break problem into two cases
- Simple/base case: if n<10
  - Simply write number n to screen
- Recursive case: if n>=10, two subtasks:

  1- Output all digits except last digit

  2- Output last digit
- Example: argument 1234:
  - 1st subtask displays 1, 2, 3 vertically
  - 2nd subtask displays 4

writeVertical Function Definition

- Given previous cases:

```
void writeVertical(int n)
{
    if (n < 10)                    //Base case
        cout << n << endl;
    else
    {                              //Recursive step
        writeVertical(n/10);
        cout << (n%10) << endl;
    }
}
```

writeVertical Trace

- Example call:
  writeVertical(123);
  → writeVertical(12); (123/10)
  → writeVertical(1); (12/10)
  → cout << 1 << endl;
  cout << 2 << endl;
  cout << 3 << endl;
- Arrows indicate task function performs
- Notice 1st two calls call again (recursive)
- Last call (1) displays and "ends"

Searching an Array: Binary Search (1/8)

- A high-level binary search:
  - Search a **sorted** array of integers for a given value.
    - anArray[0] <= anArray[1] <= . . . <= anArray[size-1]

```
binarySearch(anArray: ArrayType, target: ValueType)
  if (anArray is of size 1)
     Determine if anArray's value is equal to target
  else {
     Find the midpoint of anArray
     Determine which half of anArray contains target
     if (target is in the first half of anArray)
       binarySearch(first half of anArray, target)
     else
       binarySearch(second half of anArray, target)
  }
```

Searching an Array: Binary Search (2/8)

- How will you pass "half of `anArray`" to the recursive calls to `binarySearch`?
  - `binarySearch(anArray, first, last, target)`
    - Pass the entire array at each call, but have `binarySearch` search only `anArray[first..last]`.
  - The midpoint in `[first..last]` is:
    - `mid = (first + last) / 2`
  - Then `binarySearch(`*`first half of`* `anArray, value)`:

    `binarySearch(anArray, first, mid-1, target)`
  - `binarySearch(`*`second half of`* `anArray, target)`:

    `binarySearch(anArray, mid+1, last, target)`

Searching an Array: Binary Search (3/8)

- How do you determine which half of the array contains `target`?
  - The first half will be: `if (target < anArray[mid])`.
  - Remember to test the equality between `target` and `anArray[mid]`.
- What should the base case(s) be?
  - `target == anArray[mid]`
    - When `target` is in the array.
  - `first > last`
    - When `target` is not in the array.

Searching an Array: Binary Search (4/8)

- How will `binarySearch` indicate the result of the search?
  - A negative value if it does not find `target` in the array.
  - The index of the array item that is equal to `target`.

Searching an Array: Binary Search (5/8)

```
int binarySearch(const int anArray[], int first, int
  last, int target)
{
   int index;
   if (first > last)
      index = -1;          // target not in original array

   else {
     int mid = first + (last - first)/2;
     if (target == anArray[mid])
        index = mid;       // value found at anArray[mid]
```

Base cases.

Searching an Array: Binary Search (6/8)

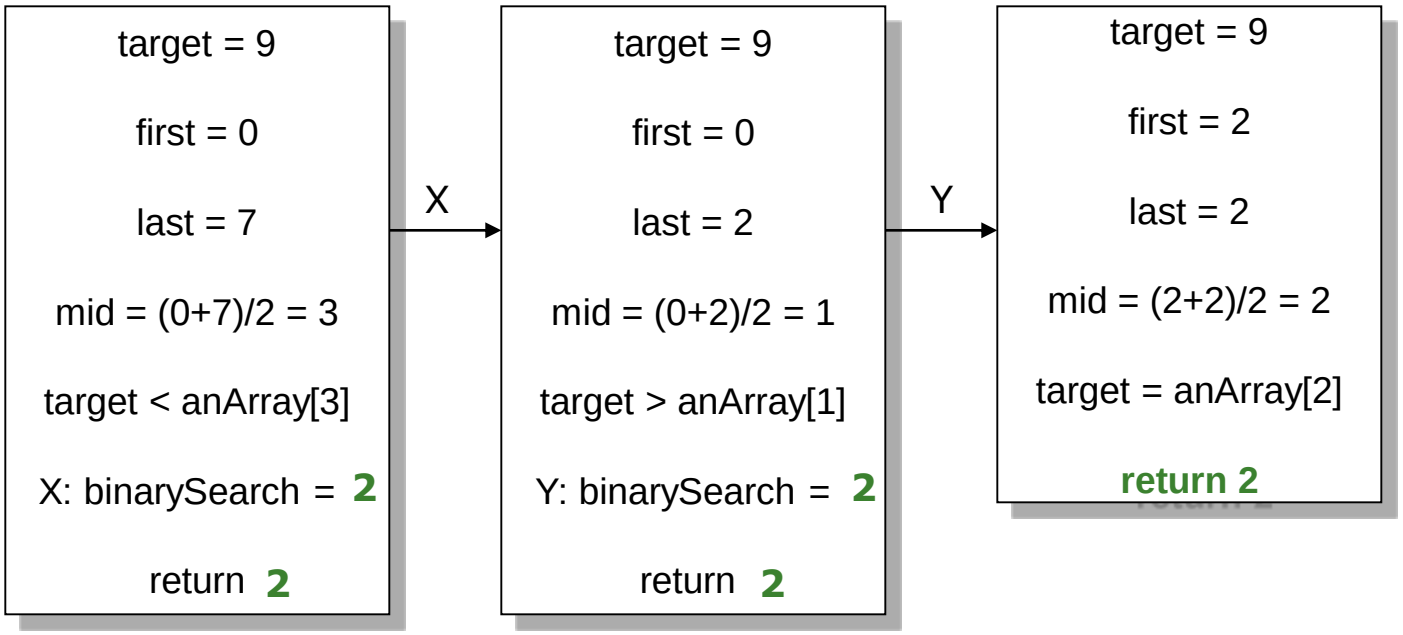
Determine which half by comparison.

```
        else if (target < anArray[mid])
            // Point X
            index = binarySearch(anArray, first, mid-1,
                                 target);
        else
            // Point Y
            index = binarySearch(anArray, mid+1, last,
                                 target);
    }  // end if
    return index;
}  // end binarySearch
```

Specify the range of the half array.

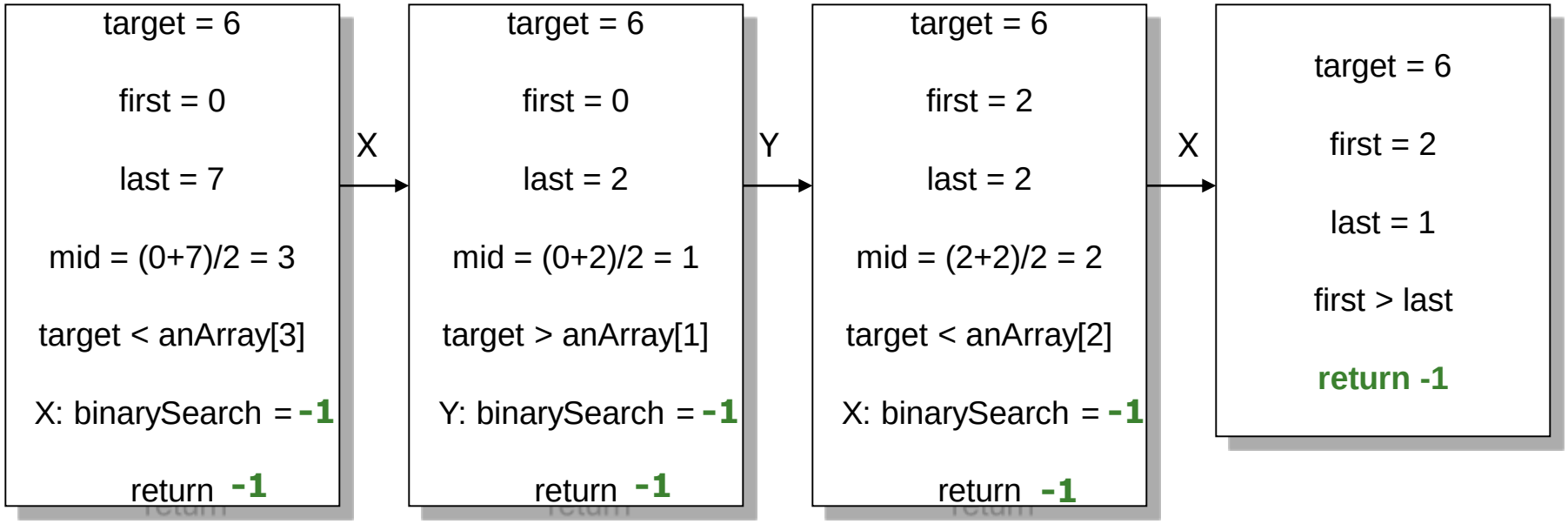
Searching an Array: Binary Search (7/8)

- Box trace of `binarySearch`:
  - `anArray = <1, 5, 9, 12, 15, 21, 29, 31>`
  - Search for `9`.



Searching an Array: Binary Search (8/8)

- Box trace of `binarySearch`:
  - `anArray = <1, 5, 9, 12, 15, 21, 29, 31>`
  - Search for `6`.



Lecture 6: Sorting Algorithms

Sorting Algorithms

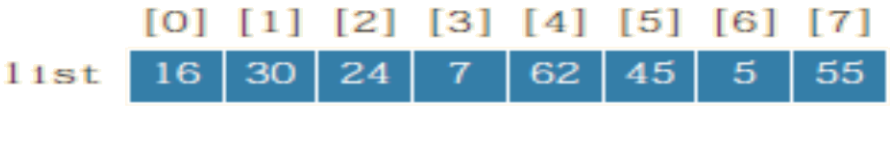
- **Sorting** refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Importance of sorting lies in the fact that data searching can be optimized to a very high level if data is stored in a sorted manner.
- Some of the examples of sorting in real life scenarios are following.
    - **Telephone Directory** − Telephone directory keeps telephone no. of people sorted on their names. So that names can be searched.
    - **Dictionary** − Dictionary keeps words in alphabetical order so that searching of any work becomes easy.

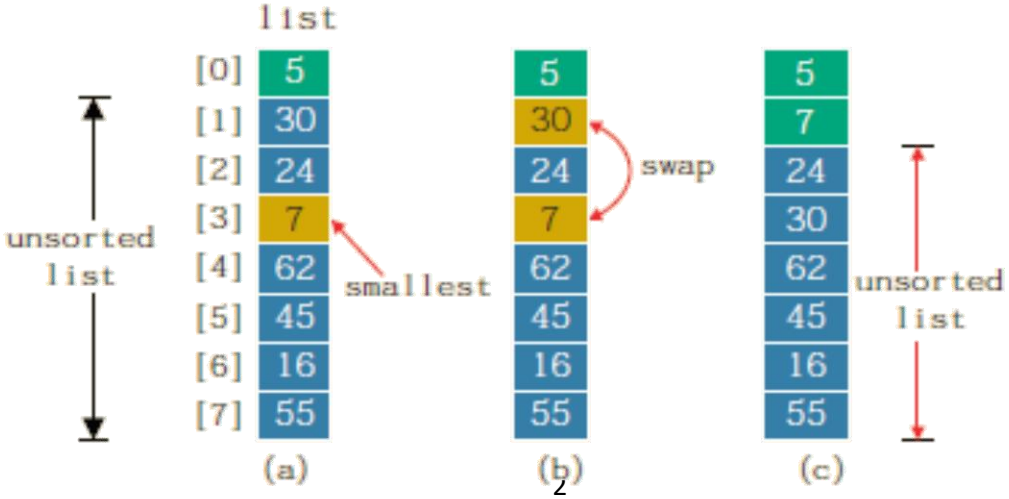
Selection Sort

- The selection sort algorithm sorts a list by selecting the smallest element in the unsorted portion of the list and then moving this smallest element to the top of the unsorted list. The first time, we locate the smallest item in the entire list; the second time, we locate the smallest item in the list starting from the second element in the list, and so on.

```
void selectionSort(elemType list[], int length)
{
    int loc, minIndex;
    for (loc = 0; loc < length; loc++)
    {
        minIndex = minLocation(list, loc, length - 1);
        swap(list, loc, minIndex);
    }
} //end selectionSort
```

Selection Sort





Bubble Sort

- In a series of n - 1 iterations, the successive elements list[index] and list[index + 1] of the list are compared. If list[index] is greater than list[index + 1], then the elements list[index] and list[index + 1] are swapped.
- It follows that the smaller elements move toward the top, and the larger elements move toward the bottom.

```
void bubbleSort(elemType list[], int length)
{
    for (int iteration = 1; iteration < length; iteration++)
    {
        for (int index = 0; index < length - iteration; index++)
        {
            if (list[index] > list[index + 1])
            {
                elemType temp = list[index];
                list[index] = list[index + 1];
                list[index + 1] = temp;
            }
        }
    }
} //end bubbleSort
```

Bubble Sort Example 1:

Illustrate the first pass of a bubble sort of the following array:

5, 1, 2, 4, 3, 7, 6

Solution:

First Pass →

5	1	2	4	3	7	6	Swap
1	5	2	4	3	7	6	Swap
1	2	5	4	3	7	6	Swap
1	2	4	5	3	7	6	Swap
1	2	4	3	5	7	6	No Swap
1	2	4	3	5	7	6	Swap
1	2	4	3	5	6	7	

Bubble Sort Example 2:

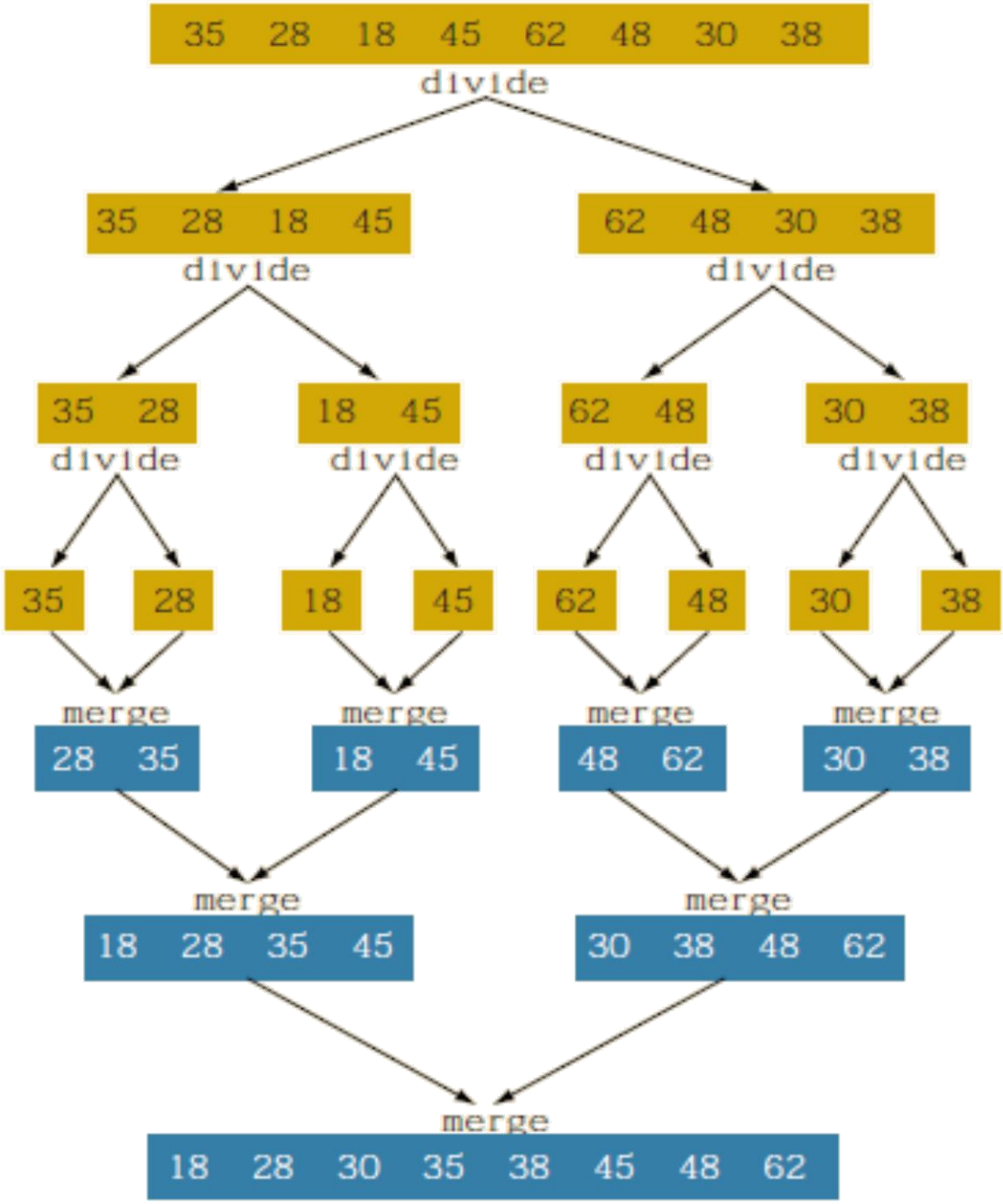
Illustrate the first two passes of a bubble sort of the following array.

The first two passes of a bubble sort of an array of five integers

	(a) Pass 1					(b) Pass 2				
Initial array:	29	10	14	37	13	10	14	29	13	**37**
	10	29	14	37	13	10	14	29	13	**37**
	10	14	29	37	13	10	14	29	13	**37**
	10	14	29	37	13	10	14	13	**29**	**37**
	10	14	29	13	**37**					

Merge Sort

- The merge sort algorithm uses the divide-and-conquer technique to sort a list. Merge sort first divides the array into equal halves and then combines them in a sorted manner.



Merge Sort Example 2

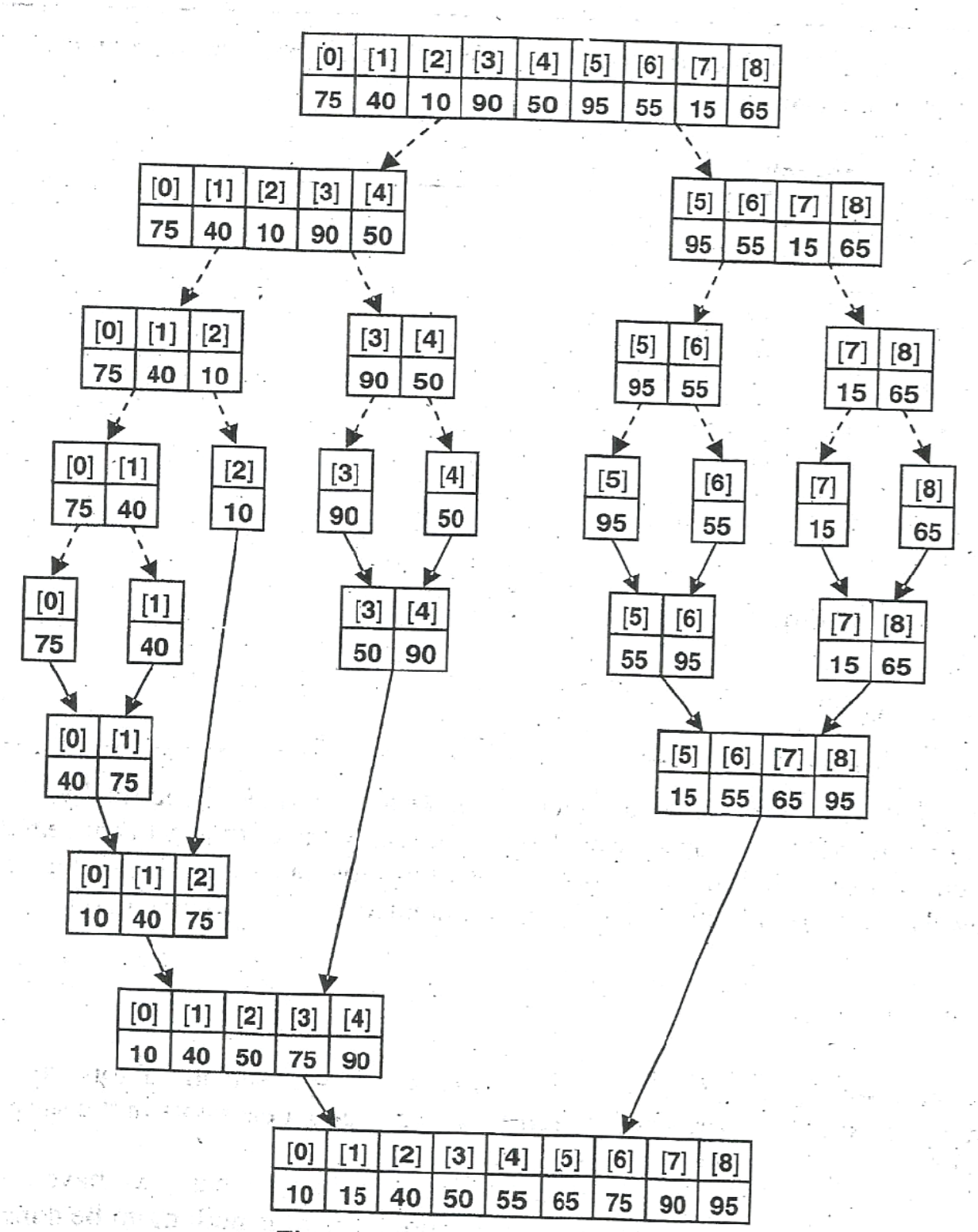
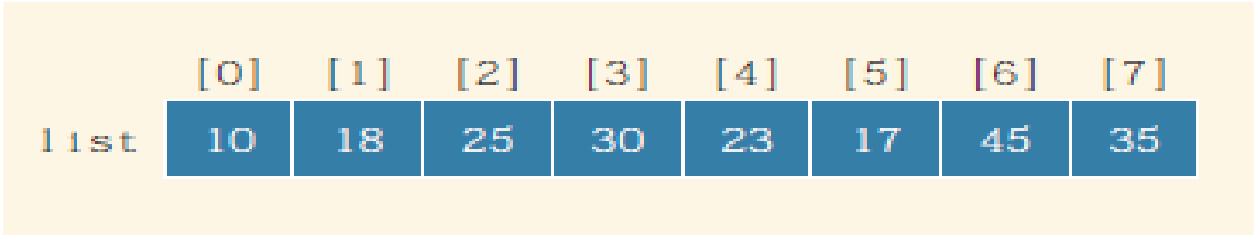


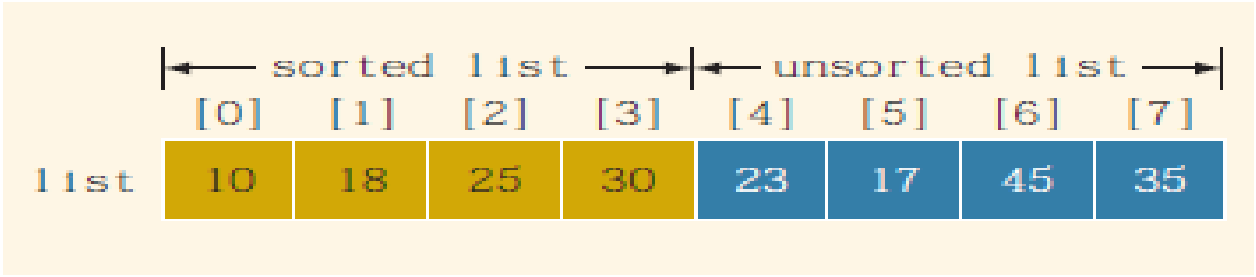
Figure 12.1: Two-way merge sort

Insertion Sort

- The insertion sort algorithm sorts the list by **moving** each element to its **proper place** in the **sorted portion** of the list.

```
void insertionSort(elemType list[], int length)
{
    for (int firstOutOfOrder = 1; firstOutOfOrder < length; firstOutOfOrder++)
        if (list[firstOutOfOrder] < list[firstOutOfOrder - 1])
        {
            elemType temp = list[firstOutOfOrder];
            int location = firstOutOfOrder;
            do
            {
                list[location] = list[location - 1];
                location--;
            }
            while (location > 0 && list[location - 1] > temp);
            list[location] = temp;
        }
} //end insertionSort
```





Insertion Sort

- Insertion sort consists of $N-1$ **passes.**
- For pass $p=1$ through $N-1$, insertion sort ensures that the elements in positions 0 through p are in sorted order.
- Insertion sort makes use of the fact that elements in positions 0 through $p-1$ are already known to be in sorted order.
- Figure 7.1 shows a sample array after each pass of insertion sort.
- Figure 7.1 shows the general strategy. In pass p, we move the element in position p left until its correct place is found among the first $p+1$ elements.
- Data movement implementation: The element in position p is moved to tmp, and all larger elements (prior to position p) are moved one spot to the right. Then tmp is moved to the correct spot.

Original	34	8	64	51	32	21	Positions Moved
After $p = 1$	8	34	64	51	32	21	1
After $p = 2$	8	34	64	51	32	21	0
After $p = 3$	8	34	51	64	32	21	1
After $p = 4$	8	32	34	51	64	21	3
After $p = 5$	8	21	32	34	51	64	4

Figure 7.1 Insertion sort after each pass

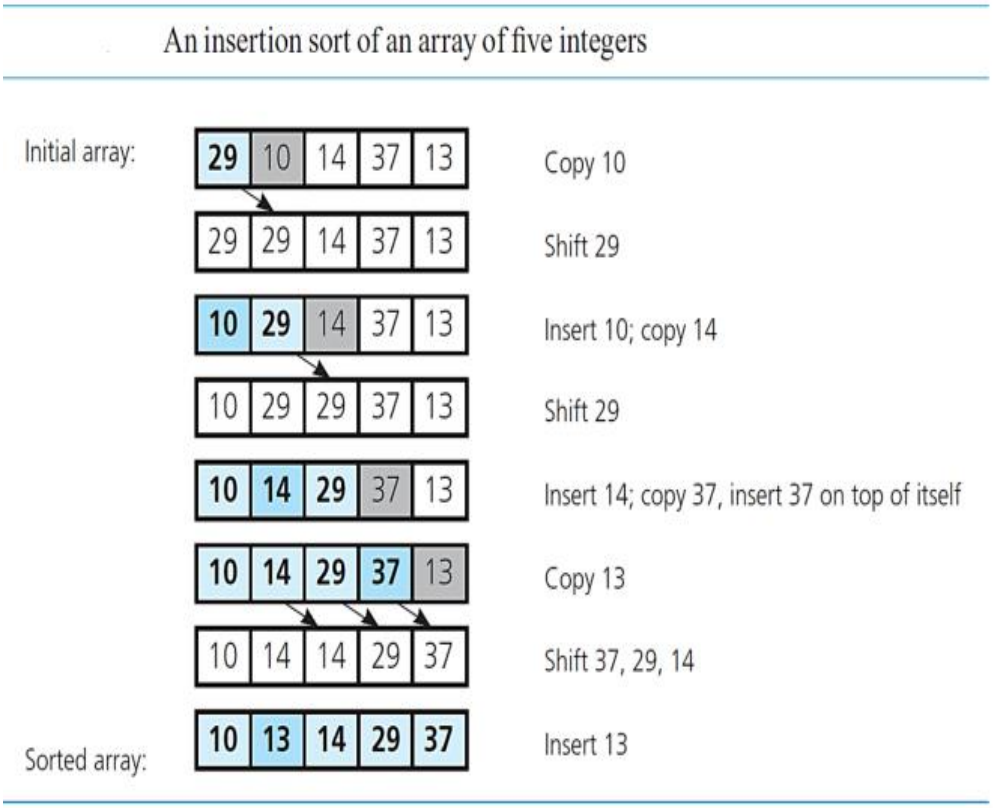
Insertion Sort Examples:

Example 1: Original Array: 29, 10, 14, 37, 13

Solution:

Original	29	10	14	37	13	**Positions Moved**
After p=1	10	29	14	37	13	1
After p=2	10	14	29	37	13	1
After p=3	10	14	29	37	13	0
After p=4	10	13	14	29	37	3

Illusration of how it is implemented:



Example 2: Original Array: (5, 6, 2, 4, 7, 3, 1).

Solution:

Original	5	6	2	4	7	3	1	**Positions Moved**
After p=1	5	6	2	4	7	3	1	**0**
After p=2	2	5	6	4	7	3	1	**2**
After p=3	2	4	5	6	7	3	1	**2**
After p=4	2	4	5	6	7	3	1	**0**
After p=5	2	3	4	5	6	7	1	**4**
After p=6	1	2	3	4	5	6	7	**6**

5	6	2	4	7	3	1	Insert 5
5	6	2	4	7	3	1	Insert 6
2	5	6	4	7	3	1	Insert 2
2	4	5	6	7	3	1	Insert 4
2	4	5	6	7	3	1	Insert 7
2	3	4	5	6	7	1	Insert 3
1	2	3	4	5	6	7	Insert 1

Quick Sort

- The quick sort algorithm uses the **divide-and-conquer** technique to sort a list.
- The list **is partitioned into two sublists**, and **combined** into one list in such a way so that the combined list is sorted.

```
void recQuickSort(elemType list[], int first, int last)
{
    int pivotLocation;
    if (first < last)
    {
        pivotLocation = partition(list, first, last);
        recQuickSort(list, first, pivotLocation - 1);
        recQuickSort(list, pivotLocation + 1, last);
    }
} //end recQuickSort
```

```
void quickSort(elemType list[], int length)
{
    recQuickSort(list, 0, length - 1);
} //end quickSort
```

```
int partition(elemType list[], int first, int last)
{
    elemType pivot;
    int index, smallIndex;
    swap(list, first, (first + last) / 2);
    pivot = list[first];
    smallIndex = first;
    for (index = first + 1; index <= last; index++)
        if (list[index] < pivot)
        {
            smallIndex++;
            swap(list, smallIndex, index);
        }
        swap(list, first, smallIndex);
        return smallIndex;
} //end partition
```

Data Structure

collage of Computer Engineering

Second Class

Algorithm Analysis

Algorithms

- **Algorithm** is a step by step procedure, which defines a set of instructions to be executed in certain order to get the desired output.
- **Algorithm Analysis:** Efficiency of an algorithm can be analyzed at two different stages,
  - A priori analysis – This is theoretical analysis of an algorithm. Efficiency of algorithm is measured by assuming that all other factors e.g. processor speed, are constant and have no effect on implementation.
  - A posterior analysis – This is empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required, are collected.

Algorithm Complexity

- two main factors which decide the efficiency of the algorithm:
  - **Time Factor –** The time is measured by counting the number of key operations such as comparisons in sorting algorithm. Time requirements can be defined as a numerical function T(n), where T(n) can be measured as the number of steps, provided each step consumes constant time.
  - **Space Factor –** The space is measured by counting the maximum memory space required by the algorithm.

Big Oh Notation, O

- A convenient way of describing the growth rate of a function and hence the time complexity of an algorithm.

Operators

Because each machine instruction can be executed in a fixed number of cycles, we may assume each operation requires a fixed number of cycles

- The time required for any operator is $O(1)$ including:
  - Retrieving/storing variables from memory
  - Variable assignment `=`
  - Integer operations `+ - * / % ++ --`
  - Logical operations `&& || !`
  - Bitwise operations `& | ^ ~`
  - Relational operations `== != < <= => >`

Blocks of Operations

Each operation runs in $O(1)$ time and therefore any fixed number of operations also run in $O(1)$ time, for example:

```
// Swap variables a and b
int tmp = a;
a = b;
b = tmp;
```

Control Statements

These are statements which potentially alter the execution of instructions

- Conditional statements

  `if`, **`switch`**

- Condition-controlled loops

  `for`, **`while`**, **`do-while`**

- Count-controlled loops

```
for i from 1 to 10 do ... end do;        # Maple
```

- Collection-controlled loops

```
foreach ( int i in array ) { ... }       // C#
```

Control Statements

Given

```
if ( condition ) {
    // true body
} else {
    // false body
}
```

The run time of a conditional statement is:

- the run time of the condition (the test), plus
- the run time of the body which is run

- Loop For example,

```
for ( int i = 0; i < n; ++i ) {
    // ...
}
```

Assuming there are no break or return statements in the loop, the run time is $O(n)$

- If the body does not depend on the variable (in this example, `i`), then the run time of

```
for ( int i = 0; i < n; ++i ) {
    // code which is Theta(f(n))
}
```

is $O(n\ f(n))$

Blocks in Sequence

Suppose you have now analyzed a number of blocks of code run in sequence

```
template <typename T>
void update_capacity( int delta ) {
   T *array_old = array;
   int capacity_old = array_capacity;
   array_capacity += delta;
   array = new T[array_capacity];

   for ( int i = 0; i < capacity_old; ++i ) {
     array[i] = array_old[i];
   }
}
```

$O(1)$

$O(n)$

To calculate the total run time, add the entries: $O(1 + n) = O(n)$

Example 1

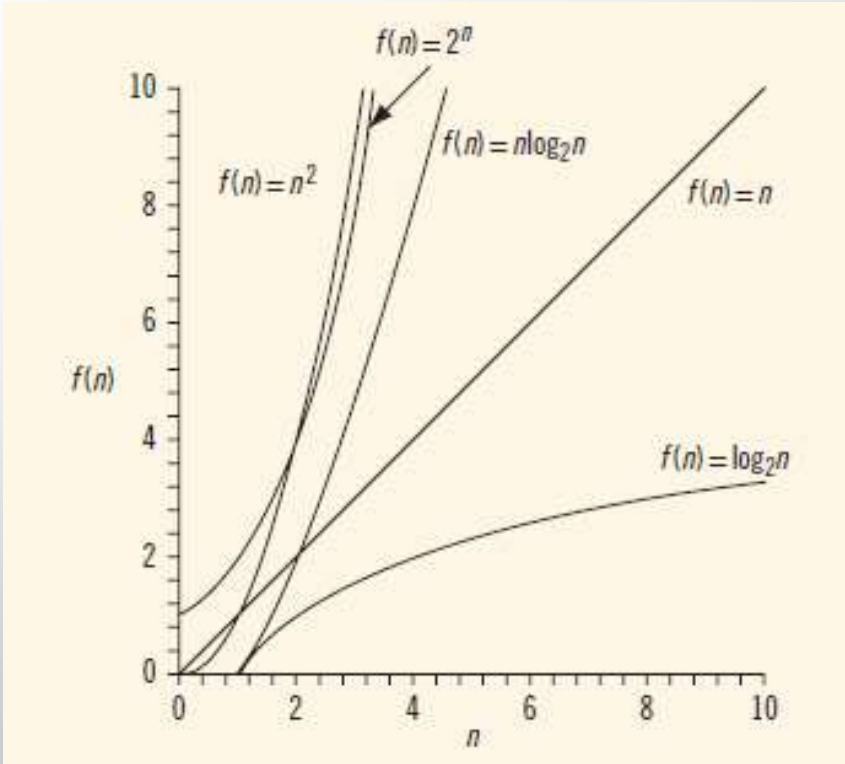
```
int sum = 0;
        for ( int i = 0; i < n; ++i ) {
            for ( int j = 0; j < n; ++j ) {
                sum += 1; }
        }
```

The previous example showed that the inner loop is $O(n)$, thus the outer loop is

$O(n \cdot n) = O(n^2)$

Growth Rate of Various Functions

n	$\log_2 n$	$n\log_2 n$	n^2	2^n
1	0	0	1	2
2	1	2	2	4
4	2	8	16	16
8	3	24	64	256
16	4	64	256	65536
32	5	160	1024	4294967296



Function $g(n)$	Growth rate of $f(n)$
$g(n) = 1$	The growth rate is constant, so it does not depend on n, the size of the problem.
$g(n) = \log_2 n$	The growth rate is a function of $\log_2 n$. Because a logarithm function grows slowly, the growth rate of the function f is also slow.
$g(n) = n$	The growth rate is linear. The growth rate of f is directly proportional to the size of the problem.
$g(n) = n\log_2 n$	The growth rate is faster than the linear algorithm.
$g(n) = n^2$	The growth rate of such functions increases rapidly with the size of the problem. The growth rate is quadrupled when the problem size is doubled.
$g(n) = 2^n$	The growth rate is exponential. The growth rate is squared when the problem size is doubled.

Algorithm Analysis

How to Determine Big O (The Rules)

When analyzing code, you follow two main logical steps to simplify the expression:

Drop the Constants

Big O ignores fixed multipliers because we care about the rate of growth, not the exact number of operations.

Example: If an algorithm takes 2n+5 operations, we ignore the +5 and the 2 to say it is O(n).

Keep the Dominant Term

If an algorithm has multiple parts, you only keep the part that grows the fastest as n becomes very large.

- **Example:** For an algorithm with complexity n^2+n+1, the n^2 term will eventually dwarf the others. Therefore, the complexity is **$O(n^2)$**

Rule:

If $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then

(a) $T_1(N) + T_2(N) = O(f(N) + g(N))$ (intuitively and less formally it is $O(\max(f(N), g(N)))$),

(b) $T_1(N) * T_2(N) = O(f(N) * g(N))$.

Analyze the following Algorithm using (Big O Notation):

Example 1: Linear Complexity: O(n)

```
for (i=1 ; i < =n ; i++ ) ---------- n
   sum = sum + i ;  ---------------  1
```

time complexity= (1+ n)= O(n)

T(n)= n· O(1)= O(n)

Example 2:

Here is a simple program fragment to calculate $\sum_{i=1}^{N} i^3$:

```
        int sum( int n )
        {
            int partialSum;

1           partialSum = 0;
2           for( int i = 1; i <= n; ++i )
3               partialSum += i * i * i;
4           return partialSum;
        }
```

- The declarations count for no time.
- Lines 1 and 4 count for one unit each.
- Line 3 counts for four units per time executed (two multiplications, one addition, and one assignment). Its cost is O(1)
- Line 3 is executed N times.
- Line 2 cost is N
- We ignore the costs of calling the function and returning
- This function cost is O(N).

Example 3: Quadratic Complexity: $O(n^2)$

int i ;

i= 0 ; ----------1

for (i=0 ;i< n ; i++)----------n

{ print i ; }

for(j= 0 ; j < n ; j++)----------n

for (int k= 0 ; k<n ; k++)----------n

print j + k

1+n+n*n

$1+ n+ n^2 = 0(n^2)$

Note that two nested loops running to n result in n×n operations

What is the complexity of this segment of code?

```
for( i = 0; i < n; ++i )
   a[ i ] = 0;
for( i = 0; i < n; ++i )
     for( j = 0; j < n; ++j )
          a[ i ] += a[ j ] + i + j;
```

It has O(N) work followed by $O(N^2)$ work, is also $O(N^2)$

Example 4: Logarithmic Complexity: O(log n)

This occurs when the problem size is halved (or divided) in each step.

```
  int i ;
  i = 1 ; ------------------------------1
for ( int i= 1 ; i< n ; i = i * 2 )---------- log2 n
   print i;
```

Since i doubles each time, it reaches n in very few steps ($\log_2 n$) .

$1+ \log_2 n = O(\log_2 n)$

Example 5:

A recursive function where n is divided by 2 in each call: fun(n/2)

```
fun ( double n){
 if ( n < 1)  return  n ;
  print n ;
   fun (n/2)
}
```

Each call reduces n by half:

T(n)=T(n/2)+O(1)

Complexity : **O(log n)**

Example 6:

```
int    i , j ;
 for (i = 0; i < n; i++)   ---------------------- n
      for (j = 1; j < n; j = j * 3)  ----------------- log3 n
          print i + j;
```

$n * \log_3 n = O(n \log_3 n)$

Example 7:

```
for ( int i = 0 ; i< n ;i++ )------------------ n
   for ( int j= 0 ; j< n ; j++ ) ------------------ n
     for ( int  k = 1 ; k< n ; k = k * 2 )---------log2n
           print i+ j+ k  ;
```

$n^* n * \log_2 n$

$n^2 * \log_2 n = O (n^2 \log_2 n)$

Example 8:

```
 for (i = n/2; i < n; i++)                  //          n/2 → O(n)
     for (k = 1; k < n; k = k * 2)    //              O(log n)
          for (j = 1; j < n; j = j * 2) //               O(log n)
               print i + k + j;
```

$n/2 * \log_2 n * \log_2 n$

$n * \log_2 n * \log_2 n$

$n^* \log n^2 = O(n (\log_2 n)^2)$

Example 9:

While (n> 2)

-

-

n= n/2 ;

complexity = $O(\log_2 n)$

Example 10:

Recursive function:
fib :
0 1 1 2 3 5 8---------- n
Input : 0 1 2 3 4 5 6 ---------- n
Output : 0 1 1 2 3 5 8 ---------- n

```
int fib(int n ) {
  if ( n<2 ) {        ----------
return n }
return fib (n-1) + fib(n-2) ; }
```

fib (4) ---------- 1 = 2^0

fib(3) + fib (2) ---------- 2 = 2^1

fib(2) + fib(1)+ fib(1) + fib(0) ---------- 4= 2^2

fib(1)+ fib(0) + 1 + 1 + 0

1 + 0 + 1 +1+ 0

Complexity : $O(2^n)$

Example 11:

```
int fun9(int n)
{
        int i = 0, j = 0, m = 0;
        for ( i = n; i > 0; i /= 2)
                for ( j = 0; j < i; j++)
                        m += 1;
        return m;
}
```

Each time problem space is divided into half.
Time Complexity: **O(log(n))**

Example 12:

```
int fun10(int n)
{
        int i = 0, j = 0, m = 0;
        for ( i = 0; i < n; i++)
                for ( j = i; j > 0; j--)
                        m += 1;
        return m;
}
```

O(N+(N-1)+(N-2)+...) = O(N(N+1)/2) // arithmetic progression.
Time Complexity:$O(n^2)$

Example 13:

```
int fun11(int n)
{
        int i = 0, j = 0, k = 0, m = 0;
        for (i = 0; i<n; i++)
                for (j = i; j<n; j++)
                        for (k = j+1; k<n; k++)
                                m += 1;
        return m;
}
```

Time Complexity: $O(n^3)$

Example 14:

```
int fun12(int n)
{
        int i = 0, j = 0, m = 0;
        for (i = 0; i<n; i++)
                for (; j<n; j++)
                        m += 1;
        return m;
}
```

Think carefully once again before finding a solution, j value is not reset at each iteration.
Time Complexity: **O(n)**

The Tree Data Structure

A tree can be defined as collection of one or more *nodes with* a distinct node, called ***root*** *(the top node of the tree structure; a node with no parent)*; while the remaining nodes are partitioned as **(sub)trees** are connected to the root. The root of each sub-tree is said to be ***child*** of the root, and the root is the ***parent*** of each sub-tree's root.

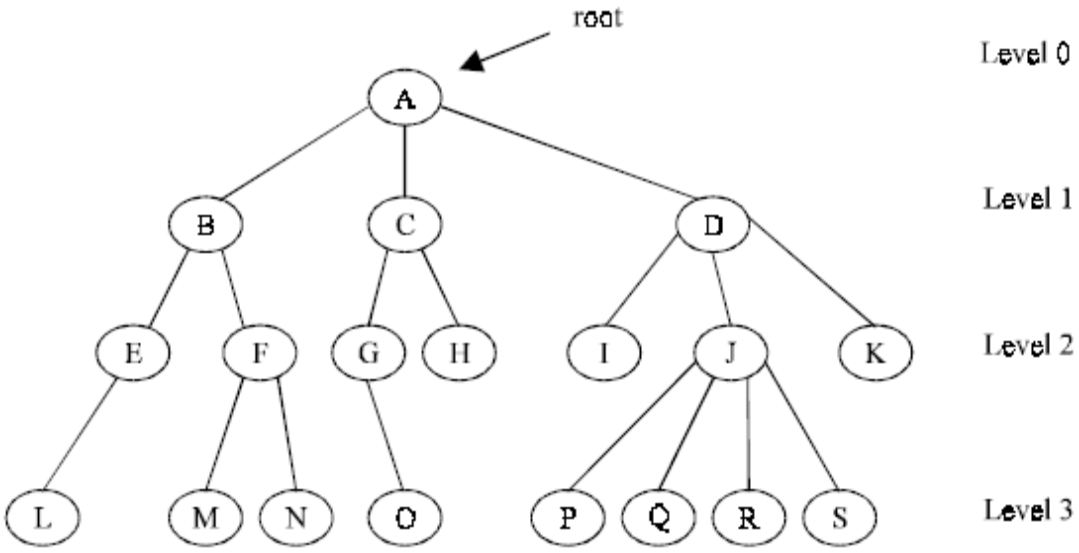


fig1:A tree

-The number subtrees of a node is called its **degree** the degree of A is 3,of B is 2. a node with degree zero is said to be *leaf* or *terminal(*a tree node that has no children) .L,M,N,O,P,Q,R and S are terminals nodes.

-nodes which are not terminal are said to be non-terminal nodes.

-The children of the same parent are said to be siblings. E and F are siblings.

-the **degree of the tree** is the degree of the maximum degree node of the tree. the degree of the tree in (fig1) is 4.

-The **ancestors** of a node are the node along the path from the node to the root.

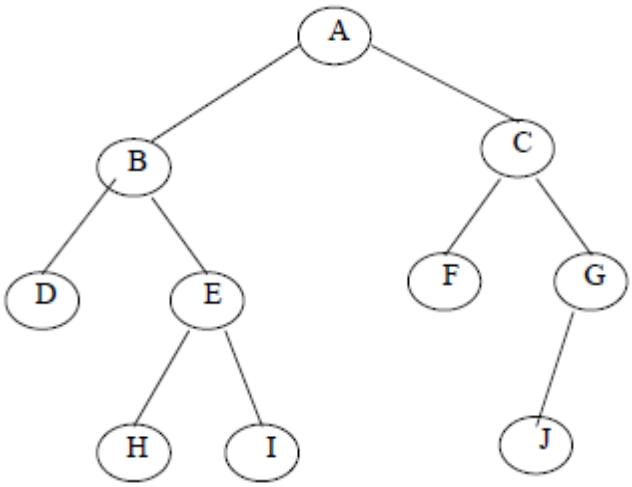
-the **level** of a node is defined recursively by assuming the level of the root to be zero, and if a node is at level *l* ,then its children are at level *l+1*.

- The **depth** or **height** of a tree is defined to be the level of a node which is maximum.

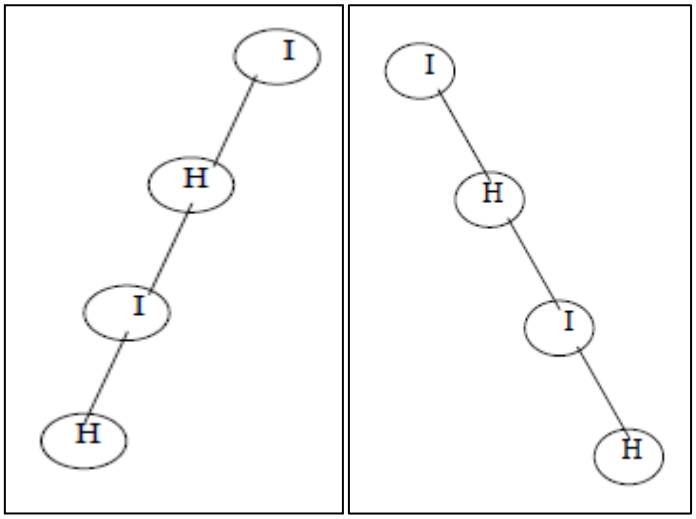
Binary Trees

A binary tree is an important tree structure which is used in several applications. It is characterized by the fact *that no node can have more than two children. Unlike trees, the binary tree distinguishes its sub-trees as left and right sub-trees.* Moreover, a binary tree may have zero nodes, whereas in trees at least one node must be present.

A binary tree is an finite set of nodes which is either empty or consists of root and two disjointed binary trees called the left sub-tree and the right sub-tree .The left and right sub-trees of a binary trees is also binary tree.



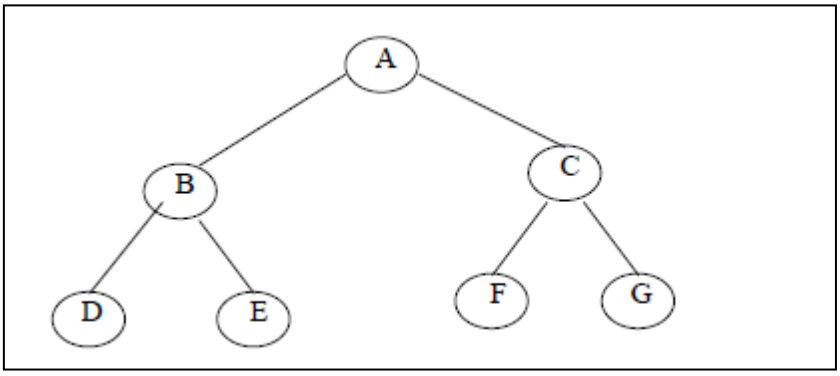
fig(2) Binary Tree



fig(3.a)left skewed binary tree fig(3.b)right skewed binary tree

there are several kind of binary tree available.

- **skewed binary tree** , in which all nodes other than the leaf nodes has only either the left or right child as in (fig 3)
- the property defined in tree is good for binary tree.
- ***trivial*** property of the binary tree is maximum number of nodes on level i of a binary tree is $\mathbf{2^i}$, i ≥1.
- if the **depth** of binary tree is k ,the maximum number of nodes in the binary tree is $\sum_{i=1}^{h} 2^{i-1} = 2^{h+1} - 1$
- **A full binary tree** is a binary tree which has the maximum number of nodes.
- in this binary tree , the nodes are **numbered** sequentially from top to bottom ,and within the levels from left to right.
- **complete binary tree** is a binary tree with n nodes and of depth k is complete if its nodes correspond to the nodes which are numbered one to n in the full binary tree on a one dimensional array.
  - the tree is **complete** if the total number of edge (**e**) in complete binary tree with (**L**) Leaves is **2(L-1)**.
  e= 2(L-1)



fig(4) complete binary tree

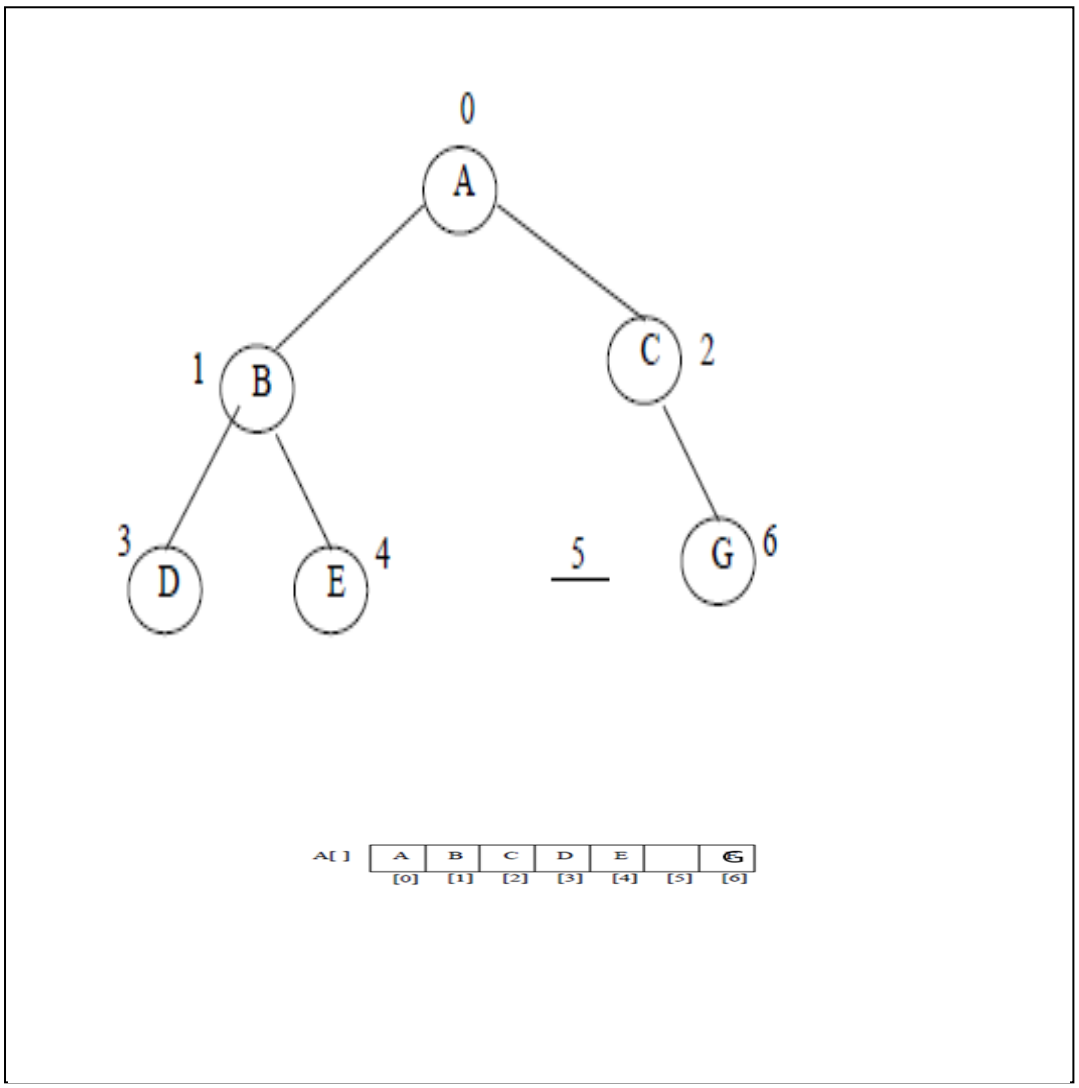
Binary Tree implementation

a) using arrays

The binary array is to be mapped into one dimensional array, BT.

in full binary tree, the node numbered *i* is being stored at BT(*i*).

- it is clear that the location of the **left** of the node stored at i is **2i+1** and the location of its **right** child is **2*i*+2.**
- we can therefore say that the location of the parent of the node at *i* is $\lfloor \boldsymbol{i - 1/2} \rfloor$
- this representation is ideal for complete binary tree as no space is wasted, some kinds of binary trees leave a lot of unused spaces when in this manner, especially in the case of skewed binary tree of depth k needs $2^k - 1$ spaces, out of which only k is really occupied.
- the sequential representation suffers during the insertion and deletion of nodes from the middle of those nodes.
- to overcome problems faced in sequential mapping , linked list can be used.



fig(5) a binary tree and its sequential mapping

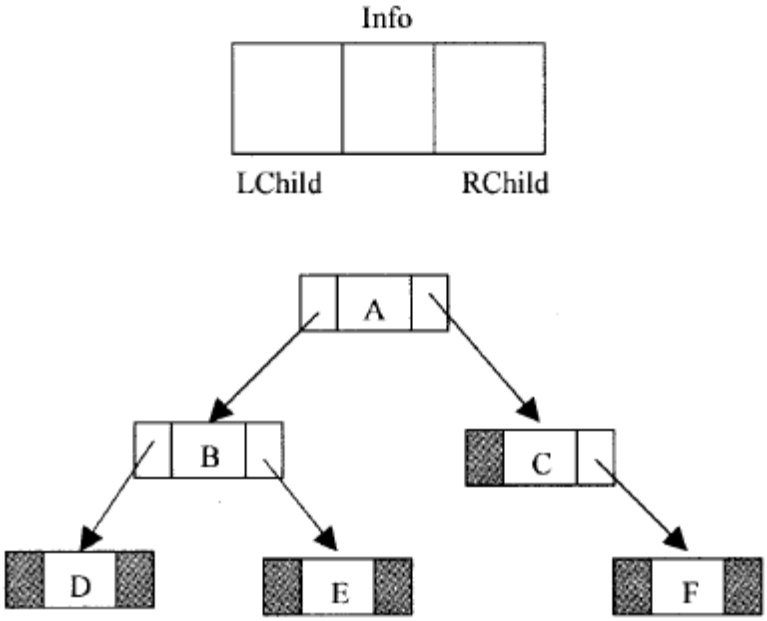
b) using linked list

pointer based implementation is more space efficient compared to sequential mapping. Here the memory must only be allocated to existing tree nodes; as each node of a binary tree can have at the most two children; the **node structure** should have at least three fields which are stated below:

a)**lchild** – the address of the left child.

b)**item**- the information of the node.

c)**rchild**- the address of the right child.



Traversal

The most important operation which can be performed on binary tree is visiting all nodes exactly once. This is said to be traversal.

-traversal can be used to find information available in the binary tree in a linear order. Since the tree is defined recursively, this recursive definition helps us to write simple recursive function for the traversal.

Binary tree traversal can be carried out in six different ways. They are:

-**Inorder**: Traversing the **left** sub-tree, visiting the **root** and finally traversing the **right** sub-tree.

-**Preorder**: visiting the **root** ,traversing the **left** sub-tree and finally traversing the **right** sub tree.

- **Postorder**: traversing the **left** sub-tree, traversing the **right** sub-tree and finally visiting the **root**.

- Reverse Inorder: Traversing the right sub-tree, visiting the root and finally traversing the right sub-tree.

- Reverse Preorder: visiting the root ,traversing the right sub-tree and finally traversing the left sub tree.

- Reverse Postorder: traversing the left sub-tree, traversing the right sub-tree and finally visiting the root.

Binary search tree

A binary search tree is a special kind of binary tree. The elements in the left and right sub-trees of each node are respectively lesser and greater than the element of that node.

Searching for an element x, in binary search tree can be performed recursively, in the following way:

1. compare x with the **root** element of the binary tree, if the binary is **non-empty**, if it matches, the element is in the ***root*** and algorithm *terminates successfully by returning the address of the root node*. if the binary tree is **empty**, it returns a **NULL** value.

2. If x is **less** than the element in the root, the search continues in the **left** sub-tree;

3. If x is **greater** than the element in the root, the search continues in the **right** sub-tree.

Level application

Trees find applications in several problems. Some of them are:

- Implementing the file systems of several operating systems.
- evaluation of arithmetic expressions.
- reduction of the time complexity of some searching operations.
- efficient sorting.
- set representation .

Binary search Tree ADT

```
#include<iostream>
#include<conio.h>
using namespace std ;
template<class T>
struct node
        {
                node *left;
                node *right;
                T data;
        };

template<class T>
class Binary_Search_Tree
{  public:
        void insertion(node<T> *&p, T x);
        void inOrderPrint(node<T> *x);
        void PreOrderPrint(node<T> *x);
        void PostOrderPrint(node<T> *x);
        node<T>* Deleting(node<T> *current, T data) ;
        node<T> *root; int count;

        Binary_Search_Tree() {root = NULL; count = 0;}
        void insert(T data) { insertion(root,data); count++; }
       void remove(T item){root= Deleting(root,item); count--;}
        void inOrderDisplay() {inOrderPrint(root); }
        void PreOrderDisplay() { PreOrderPrint(root); }
        void PostOrderDisplay() { PostOrderPrint(root); }
```

```
        node<T> *find(T key);
        int length () {return count;}
};

template<class T>
node<T>* Binary_Search_Tree<T>::Deleting(node<T> *current, T data)
{
        if(current == NULL) return current;
  if(data < current->data)
   {current->left = Deleting(current->left,data);
    return current;}
  if (data > current->data)
          {current->right = Deleting(current->right,data);
    return current; }

        //Now when we found the element
        // Case 1: No child
        if(current->left == NULL && current->right == NULL)
        {current = NULL;  return current;}

        //Case 2: One child
        if(current->left == NULL)
        {current = current->right;  return current; }
        if(current->right == NULL)
        {current = current->left;  return current;}

        // Case 3: Two children
          //Firstly we find Minimum of 'current->right'
```

```
        node<T> *min = current->right;
        while(min->left != NULL) min = min->left;
//Secondly we Copy Minimum to the node we want to delete and delete
//Minimum
        current->data = min->data;
        current->right = Deleting(current->right,min->data);
        return current;
}

template<class T>
void Binary_Search_Tree<T>::insertion(node<T> *&p,T x) //must add & (make
it by address)
{
        if(p == NULL)
        {
                p = new node<T>;
                p->left = NULL  ;
                p->right = NULL ;
                p->data = x ;
        }
        else
        {
                if(x<p->data)
                        insertion(p->left ,x);
                else
                        insertion(p->right,x);
        }
}
```

```
template<class T>
node<T>* Binary_Search_Tree<T>::find(T key)
{
	node<T> *p = root;
	if (root == NULL) return NULL;
	while (p != NULL)
	{
		if (p->data == key)
			return p;
		else if (key < p->data)
			p = p->left;
		else
			p = p->right;
	}
	return NULL;
}

template<class T>
void Binary_Search_Tree<T>::PreOrderPrint(node<T> *x)
{
	if (x)
	{
		cout << x->data << " ";
		PreOrderPrint(x->left);
		PreOrderPrint(x->right);
	}
}
```

```
template<class T>
void Binary_Search_Tree<T>::PostOrderPrint(node<T> *x)
{
	if (x)
	{
		PostOrderPrint(x->left);
		PostOrderPrint(x->right);
		cout << x->data << " ";
	}
}


template<class T>
void Binary_Search_Tree<T>::inOrderPrint(node<T> *x)
{
	if (x)
	{
		inOrderPrint(x->left);
		cout << x->data << " ";
		inOrderPrint(x->right);
	}
}

void main()
{
	Binary_Search_Tree<int> t;
t.insert(10);
```

```
t.insert(2);
t.insert(1);
t.insert(3);
t.insert(12);
t.insert(13);
t.insert(11);
t.insert(14);
cout << "In order traversal" << endl;
t.inOrderDisplay();
cout << "Pre order traversal" << endl;
t.PreOrderDisplay();
cout<< "Post order traversal" <<endl;
t.PostOrderDisplay();

Binary_Search_Tree<int> sec;

sec.remove(3);
cout<<"second tree: "<<endl;
sec.inOrderDisplay();
cout << "In order traversal" << endl;
t.inOrderDisplay();
}
```

Tree Structure

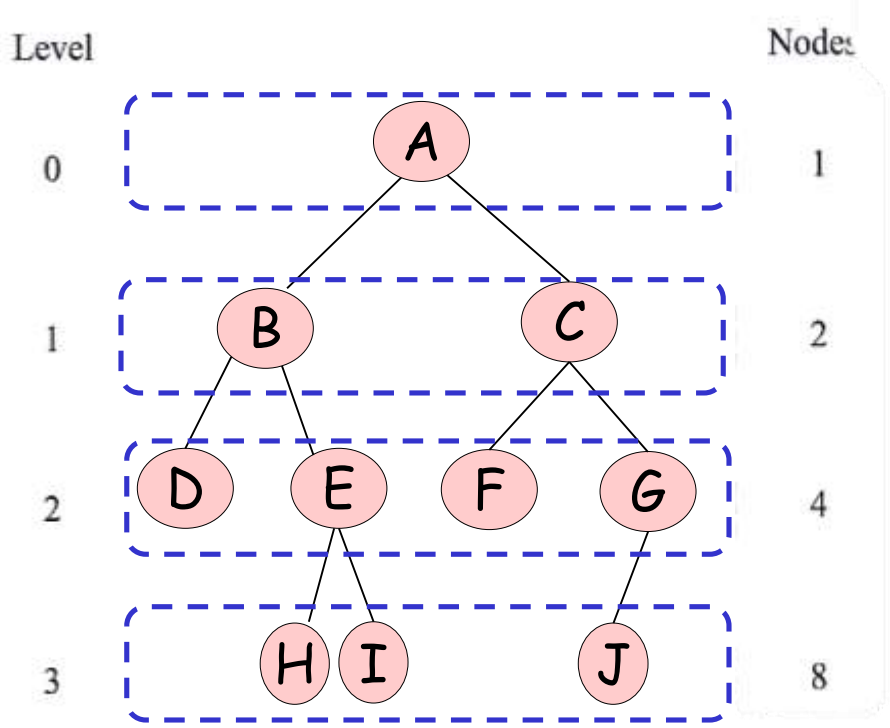
Dr. Amthal Kh. Mousa

The Tree Data Structure

Mathematical Properties

- the **level** of a node is defined recursively by assuming the level of the root to be zero, and if a node is at level ***l*** ,then its children are at level ***l+1***.

- The **depth** or **height** of a tree is defined to be the level of a node which is maximum.

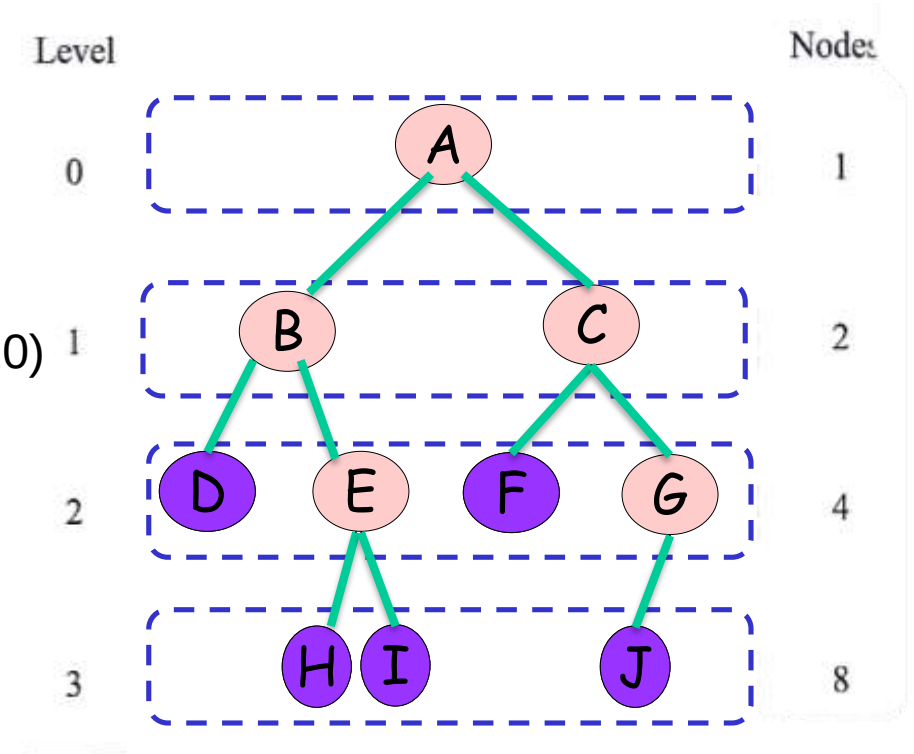


Height (h) or depth = no. of levels
= 3

Maximum number of nodes= $2^{h+1} - 1$
$= 2^{3+1} - 1$
= 15 nodes

The Tree Data Structure

Mathematical Properties



- **Is it full binary tree?**

Current Node Count: 10 nodes.

no. of nodes ≠ Maximum no. of nodes

Since the current number of nodes (10) is less than the maximum (15), it is **not a full binary tree.**

- **Is it complete binary tree?**

- the tree is **complete** if the total number of **edge** (**e**) in complete binary tree with (**L**) Leaves is **2(L-1)**.

e= 2(L-1)

e= 2(5 -1)

9 ≠ 8

it is **not a complete binary tree.**

Maximum no.of nodes= $2^{h+1} - 1$

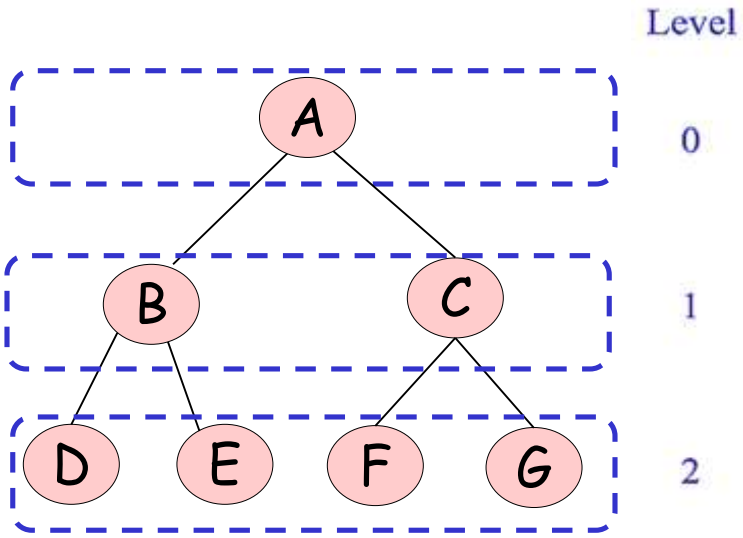
$= 2^{3+1} - 1$

= 15 nodes

. Example: A Perfect (Full) Binary Tree

A tree where all internal nodes have two children and all leaves are at the same level.

Scenario: A tree with **3 levels** (Level 0, 1, and 2).



- **Height (h):** 2.
- **Maximum Number of Nodes:**

$2^{h+1}-1=2^{2}+1-1=2^{3}-1=7$ nodes

- **Leaf Count (L):** 4 (Nodes at level 2).
- **Edge Calculation (e):**

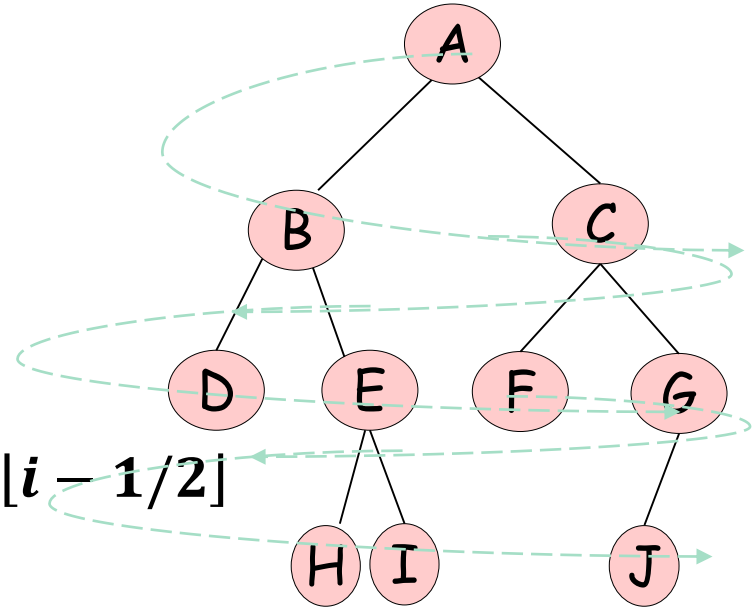
$e=2(L-1)=2(4-1)=6$ edges

Completeness Check: Since the actual edge count ($N-1=6$) matches the formula result (6), and the node count (7) matches the maximum (7), this is both a **full** and **complete** binary tree.

Binary Tree implementation or representation

(1) As arrays

- **Location** of **root** is **0**
- **left child** stored is **2i+1**
- location of **right** child is **2*i*+2.**
- For node at i, location of its parent is $\lfloor i - 1/2 \rfloor$

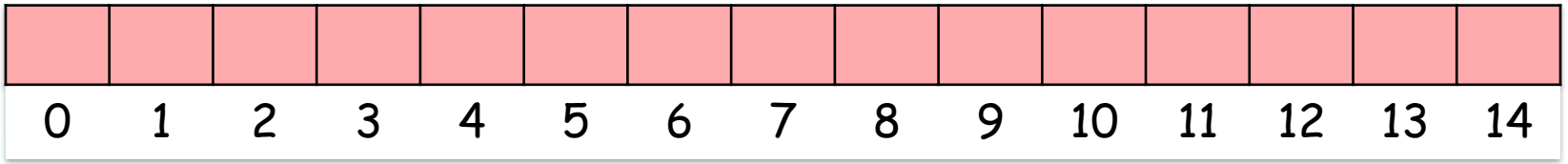


Maximum number of nodes= $2^{h+1} - 1$

$= 2^{3+1} - 1$

= 15 nodes

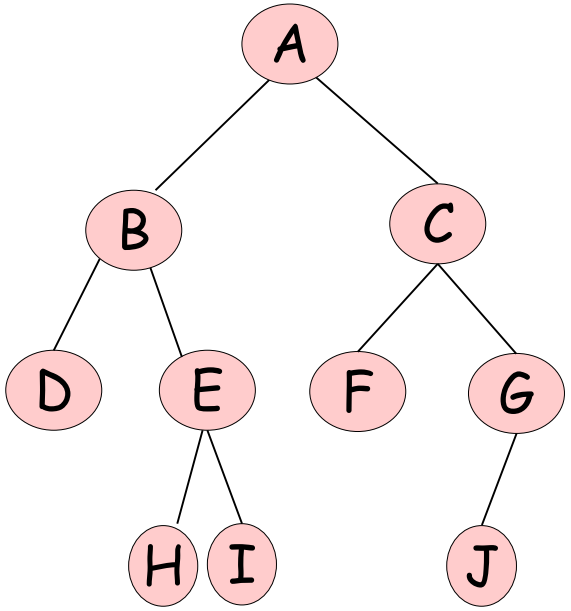
Size of array = 15



Binary Tree implementation or representation

(1) using arrays

- **Root Position:** Index 0
- **Left Child:** 2i+1
- **Right Child:** 2i+2



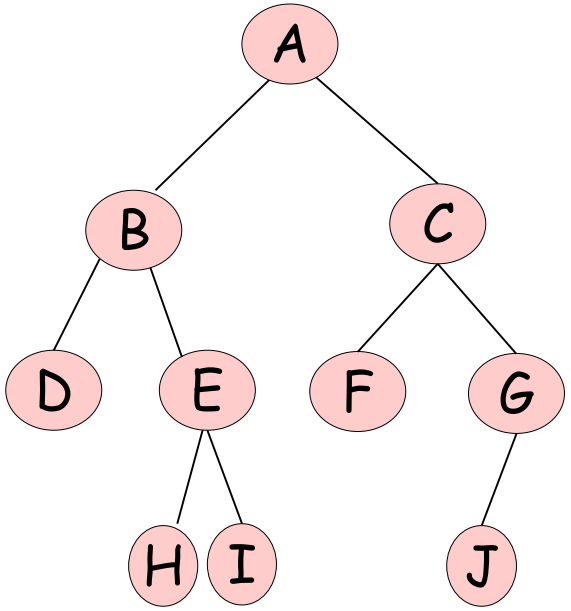
- **Node A (index 0):**
  - Left Child (B): 2(0)+1=Index 1.
  - Right Child (C): 2(0)+2=Index 2.
- **Node B (index 1):**
  - Left Child (D): 2(1)+1=Index 3.
  - Right Child (E): 2(1)+2=Index 4.

A	B	C	D	E	F	G	-	-	H	I	-	-	J	-
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Binary Tree implementation or representation

(1) using arrays

- **Root Position:** Index 0
- **Left Child:** 2i+1
- **Right Child:** 2i+2



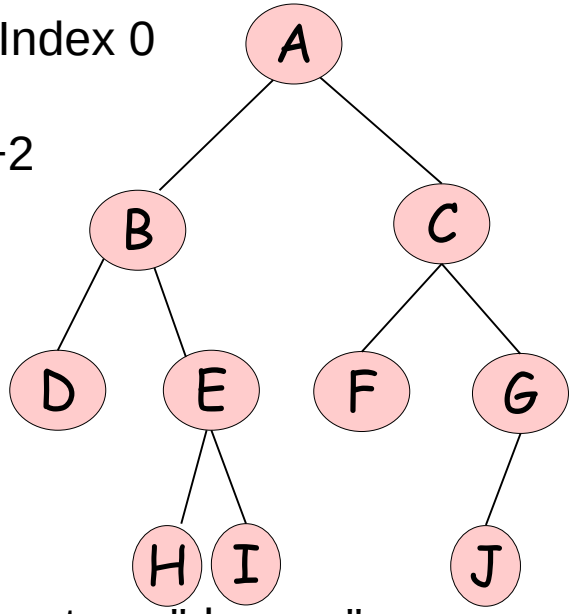
- **Node C (index 2):**
  - Left Child (F): 2(2)+1=Index 5.
  - Right Child (G): 2(2)+2=Index 6.
- **Node D (index 3):**
  - Left Child : 2(3)+1=Index 7.
  - Right Child : 2(3)+2=Index 8.
  - Has no children; indices 7 and 8 in the array remain empty or "dummy".

A	B	C	D	E	F	G	-	-	H	I	-	-	J	-
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Binary Tree implementation or representation

(1) using arrays

- **Root Position:** Index 0
- **Left Child:** 2i+1
- **Right Child:** 2i+2

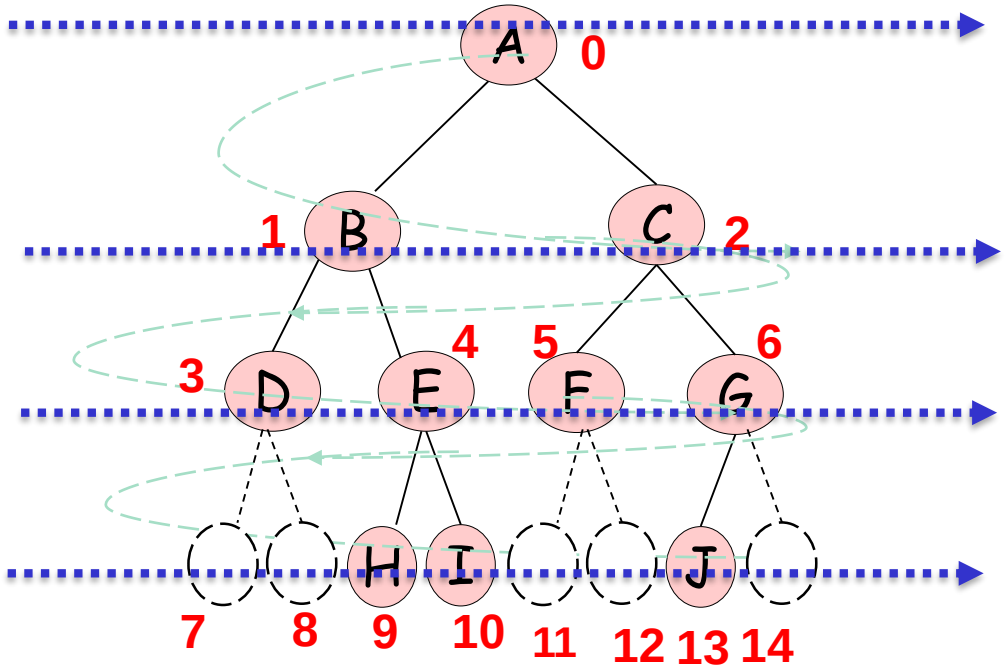


- **Node E (index 4):**
  - Left Child (H): 2(4)+1=Index 9.
  - Right Child (I): 2(4)+2=Index 10.
- **Node F (index 5):**
  - Left Child : 2(5)+1=Index 11.
  - Right Child : 2(5)+2=Index 12.
  - Has no children; indices 11 and 12 in the array remain empty or "dummy".
- **Node G(index 6):**
  - Left Child (J): 2(6)+1=Index 13.
  - Right Child : 2(6)+2=Index 14. It has no right child; index 14 remain empty or "dummy".

A	B	C	D	E	F	G	-	-	H	I	-	-	J	-
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Binary Tree implementation or representation

(1) Numbering



A	B	C	D	E	F	G	-	-	H	I	-	-	J	-
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Binary Search

- Fast way to search for elements in a **sorted array**
- Looping through elements one by one is slow [O(N)]
- Idea:

Jump to the middle element:

if the middle is what we're looking for, we're done. Hooray!

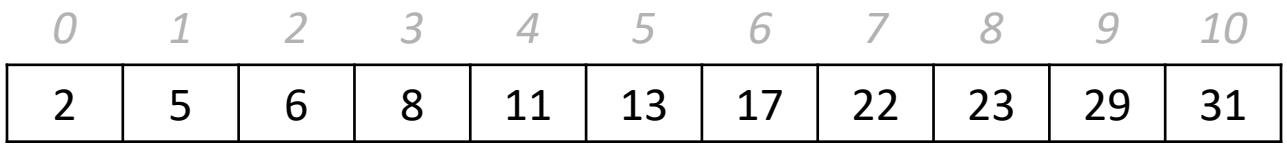
if the middle is too small – we rule out the entire **left side** of elements smaller than the middle element

if the middle is too big – we rule out the entire **right side** of elements bigger than the middle element

0	1	2	3	4	5	6	7	8	9	10
2	5	6	8	11	13	17	22	23	29	31

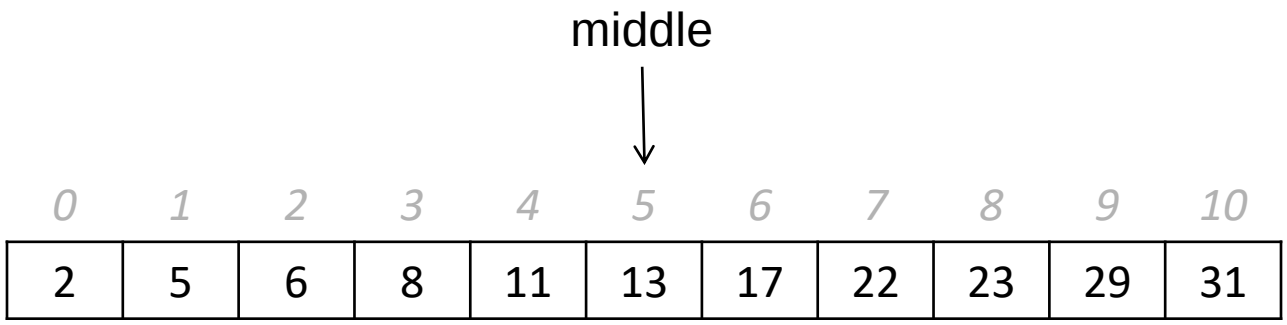
Binary Search in Action

- Search for 8:



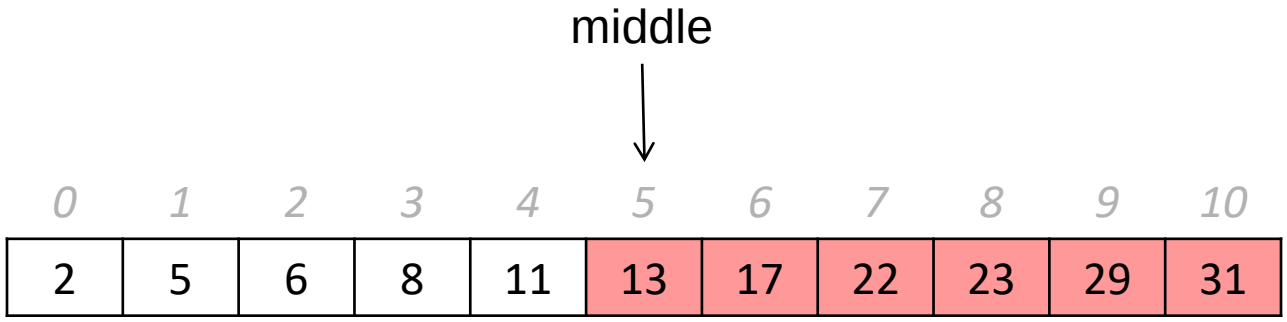
Binary Search in Action

- Search for 8:



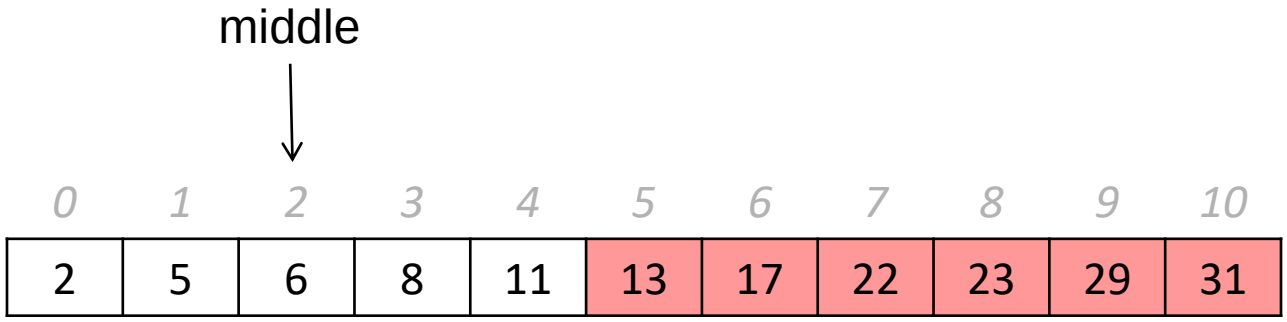
Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10



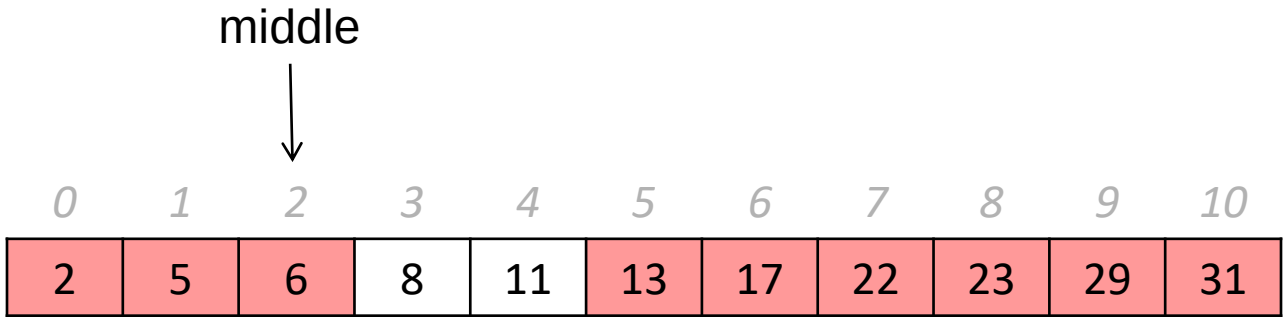
Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Pick the new middle of the remaining elements
- Look at 6:



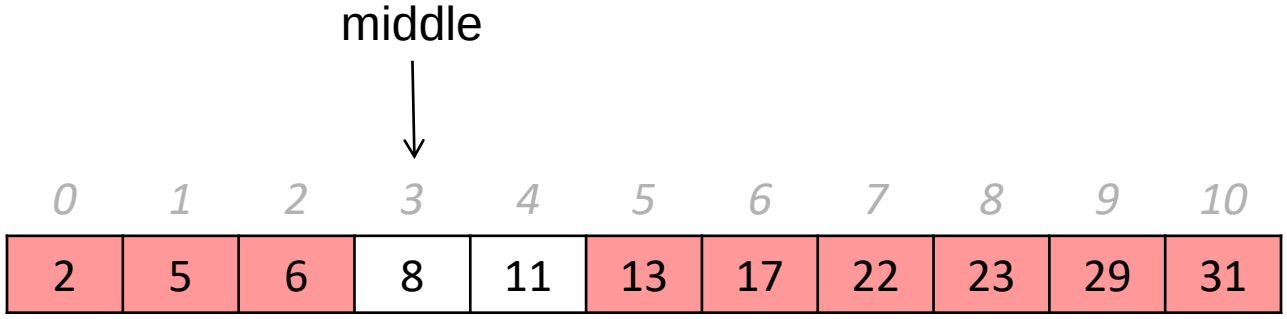
Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Pick the new middle of the remaining elements
- Look at 6:
  - it's too small, so we rule out indices 0-3



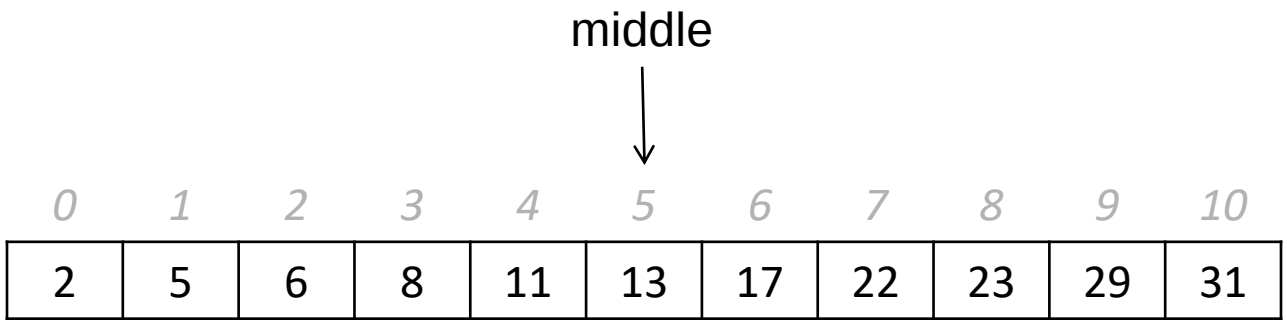
Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Pick the new middle of the remaining elements
- Look at 6:
  - it's too small, so we rule out indices 0-3
- Look at 8:
  - it's just right! We return true



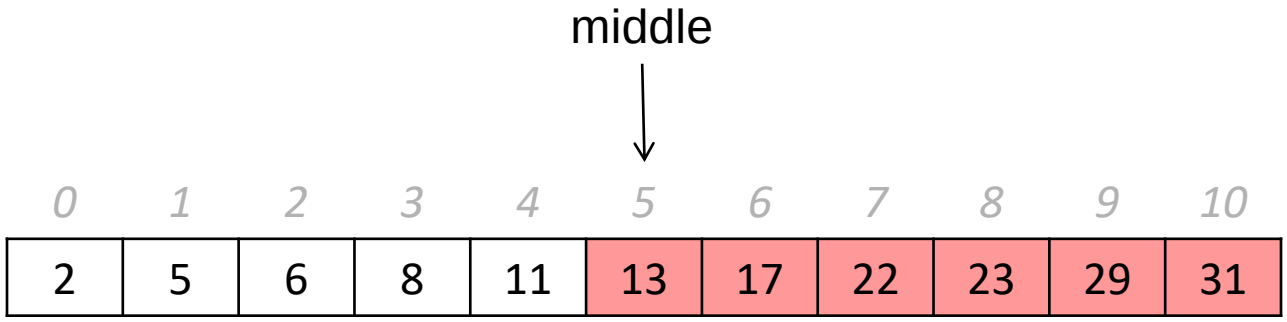
Binary Search in Action

- Search for 7:



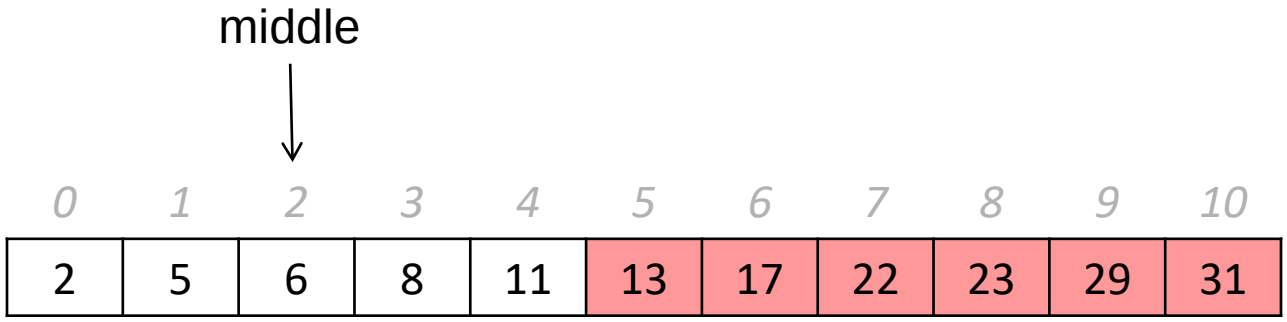
Binary Search in Action

- Search for 7:
- Look at 13
  - it's too big, so we rule out indices 5-10



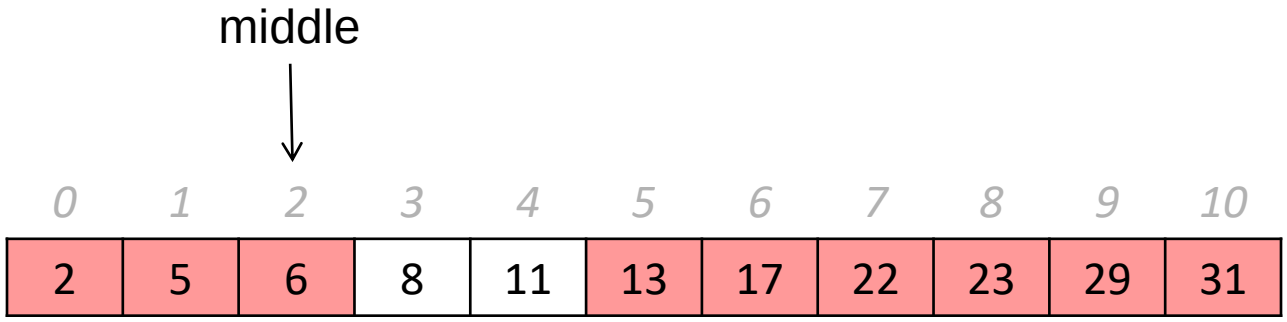
Binary Search in Action

- Search for 7:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Pick the new middle of the remaining elements
- Look at 6:



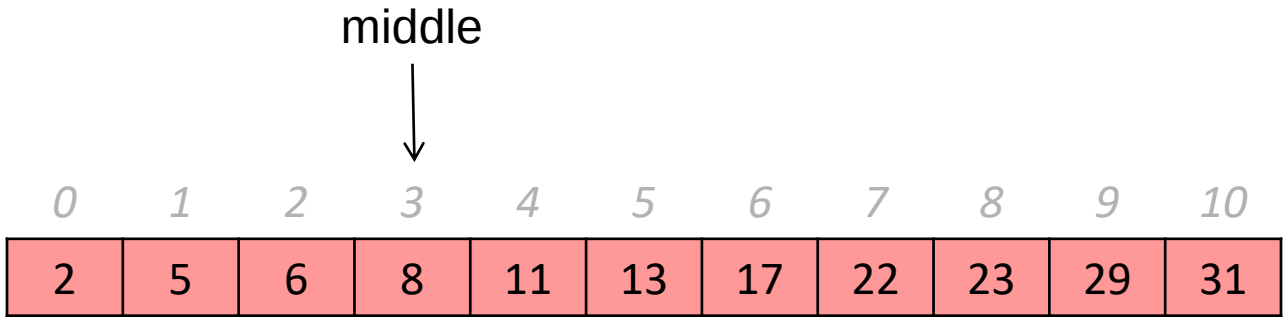
Binary Search in Action

- Search for 7:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Pick the new middle of the remaining elements
- Look at 6:
  - it's too small, so we rule out indices 0-3



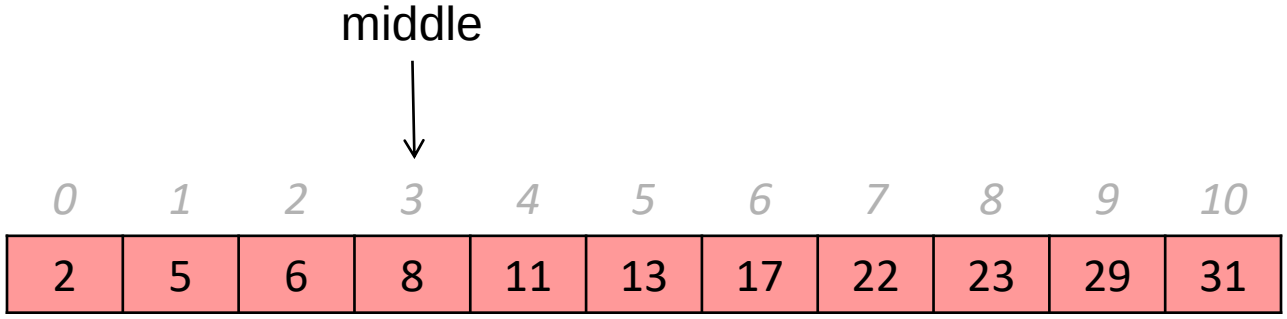
Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Look at 6:
  - it's too small, so we rule out indices 0-3
- Look at 8:
  - it's too big! We rule out elements 3-4



Binary Search in Action

- Search for 8:
- Look at 13
  - it's too big, so we rule out indices 5-10
- Look at 6:
  - it's too small, so we rule out indices 0-3
- Look at 8:
  - it's too big! We rule out elements 3-4
- No elements left to search – we return false



Binary Search Tree

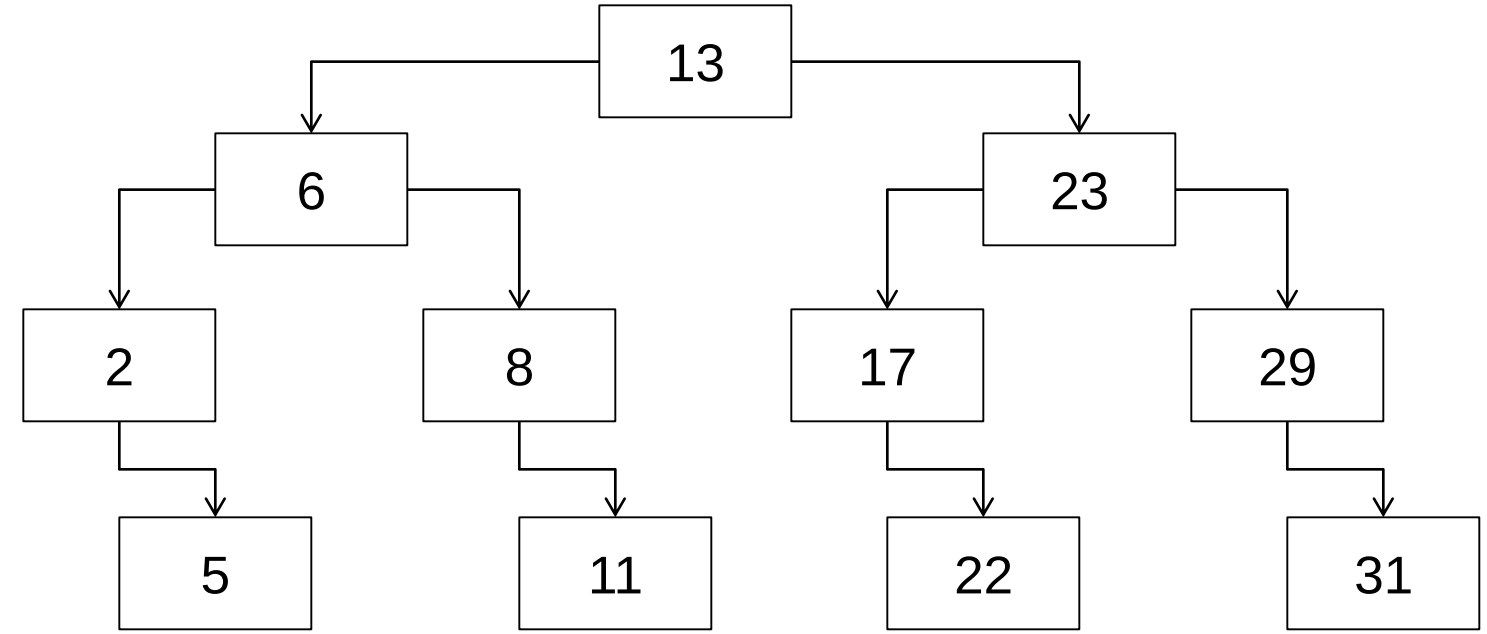
- A **tree** is a data structure where each element (**parent**) stores two or more pointers to other elements (its **children**)
  - A doubly-linked list doesn't count because, just like outside of computer science, a child can not be its own ancestor
- Each node in a **binary tree** has two pointers
  - Some of these pointers may be `nullptr` (just like in a linked list)
  - A **binary search tree** is a binary tree with special ordering properties that make it easy to do binary search
- Similar to a Linked List:
  - Each element in its own block of memory
  - Have to travel through pointers (can't skip "generations")

(Binary) TreeNode

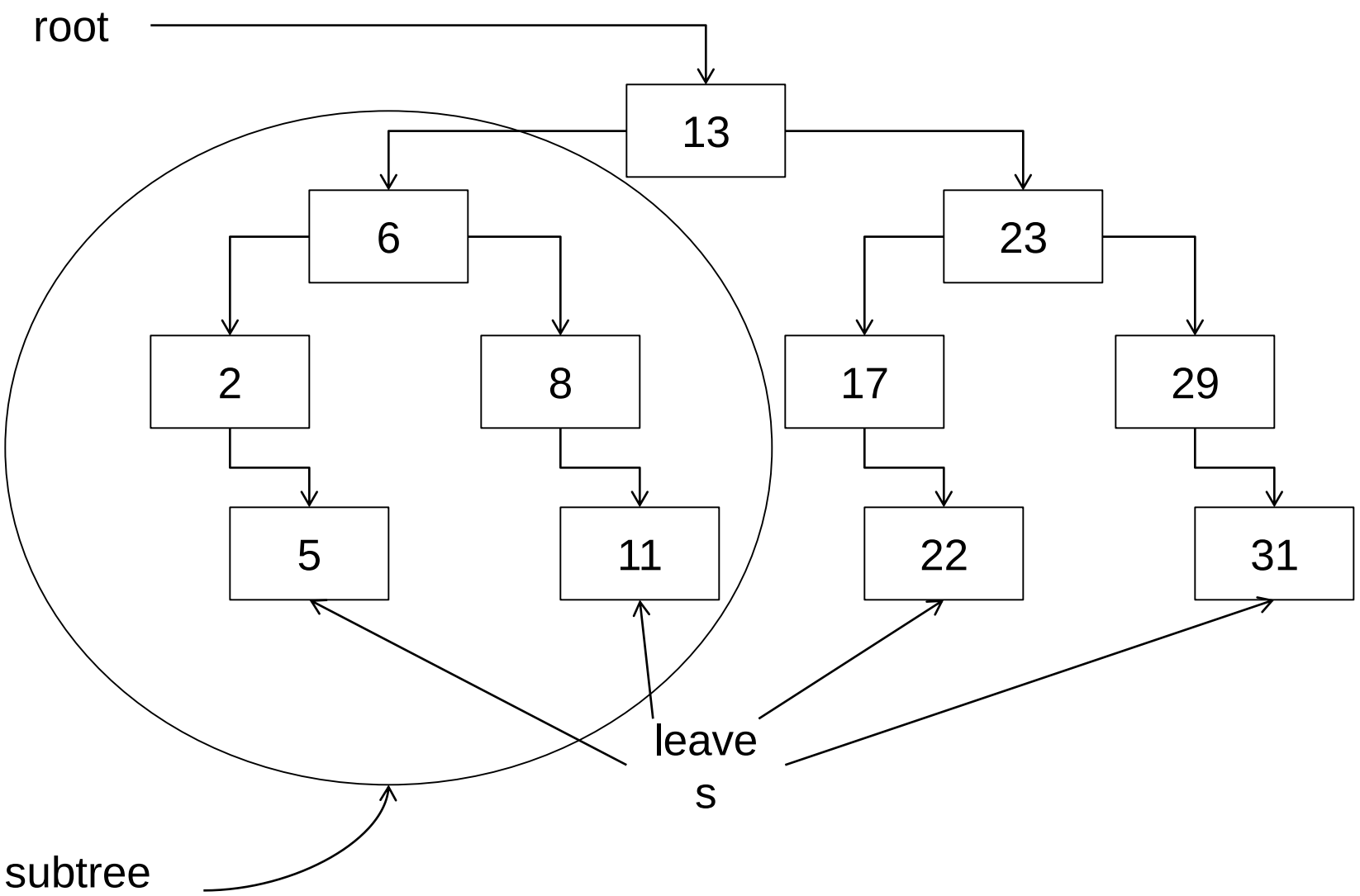
```
struct TreeNode
 {
    int data; // assume that the tree stores ints
    TreeNode *left;
    TreeNode *right;
 };
```

Binary Search Trees

- A binary search tree has the following property:
  - All elements to the **left** of an element are **smaller** than that **element**
  - All elements to the **right** of an element are **bigger** than that **element**
  - Just like our sorted array!

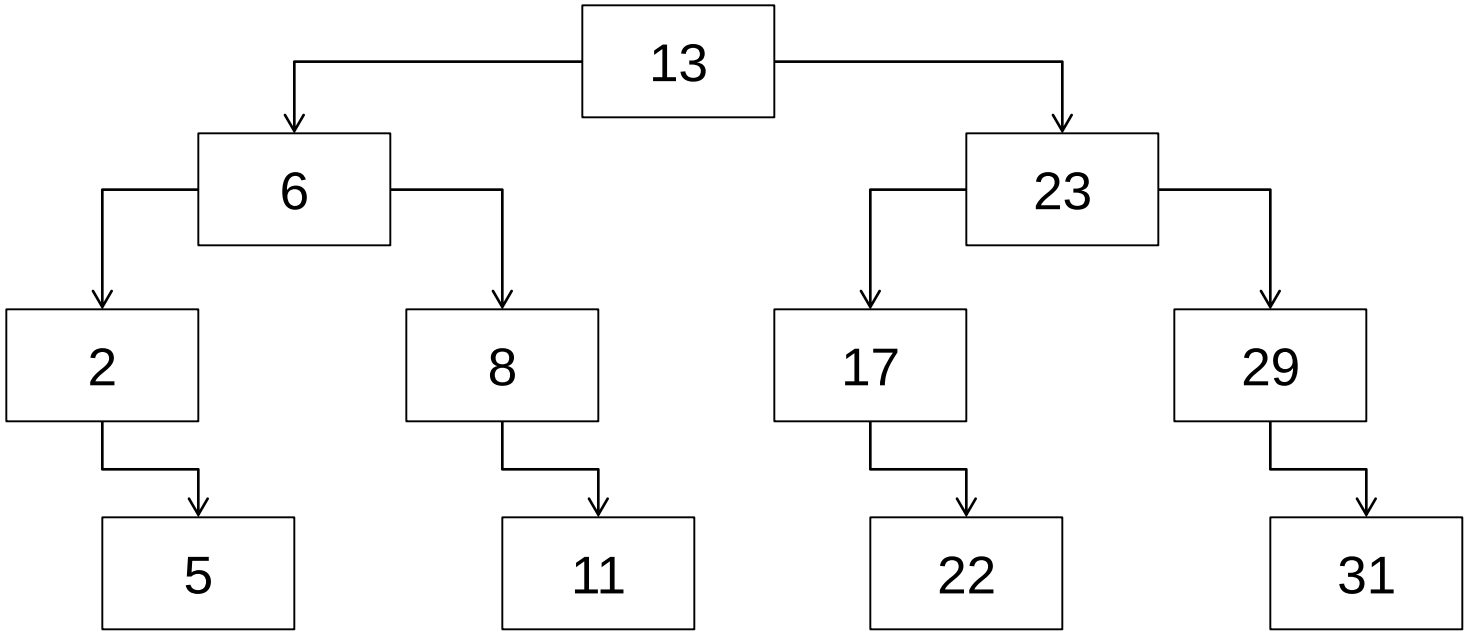


Tree anatomy



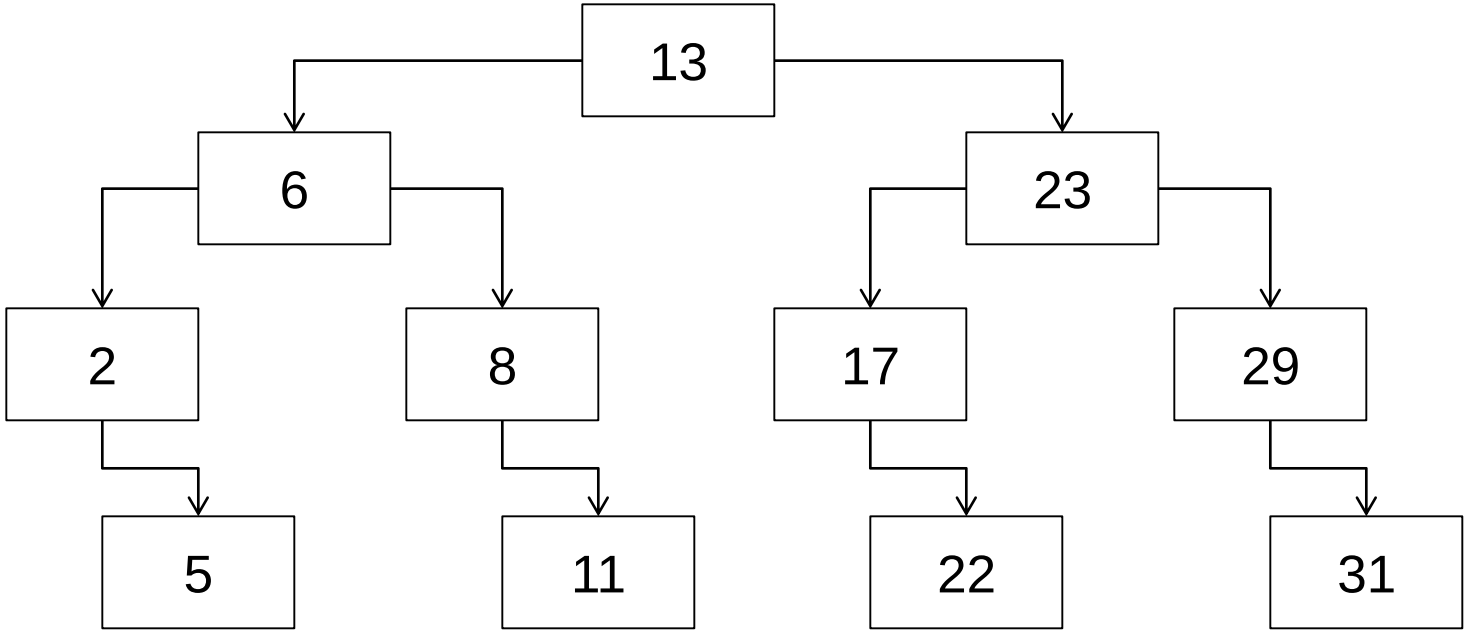
BST Contains

- How would you search a BST for an element?



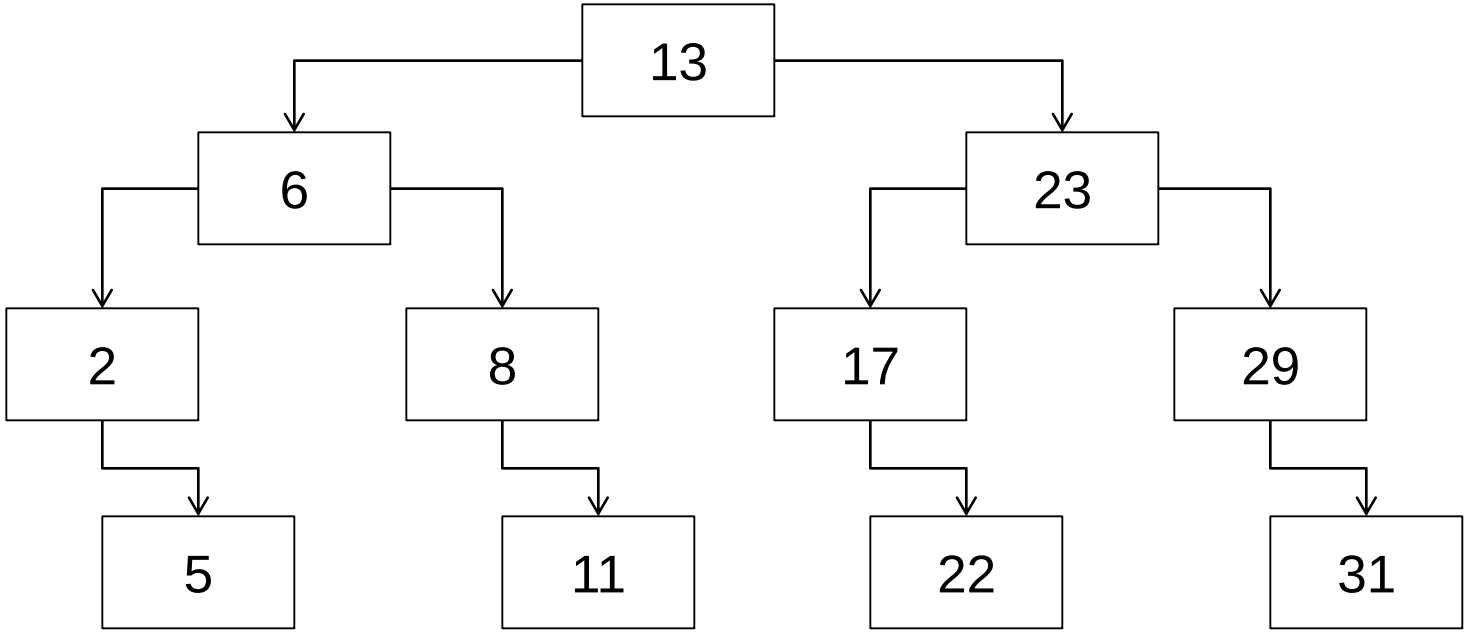
BST Contains

- How would you search a BST for an element?
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



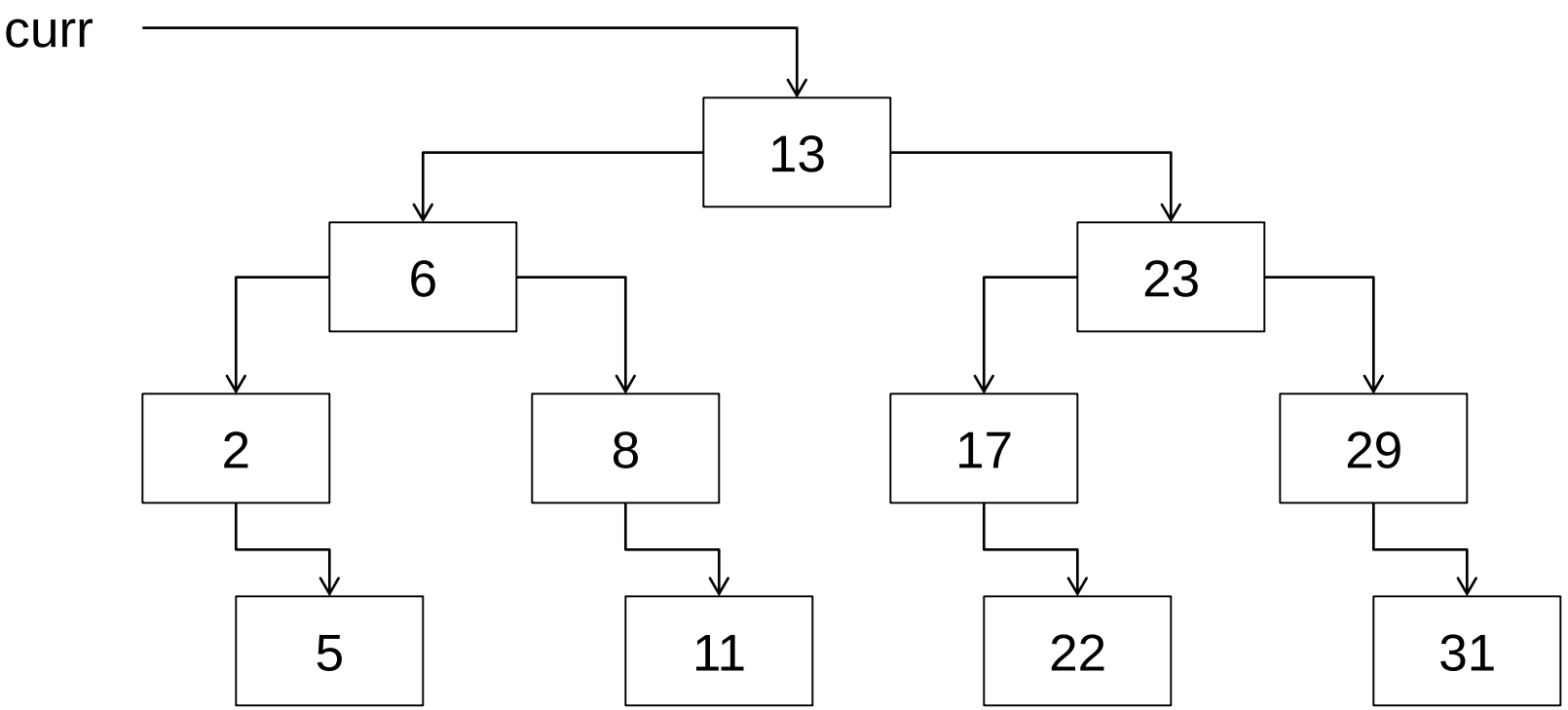
Trees and Recursion

- Trees are fundamentally **recursive (**subtrees are smaller trees)
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



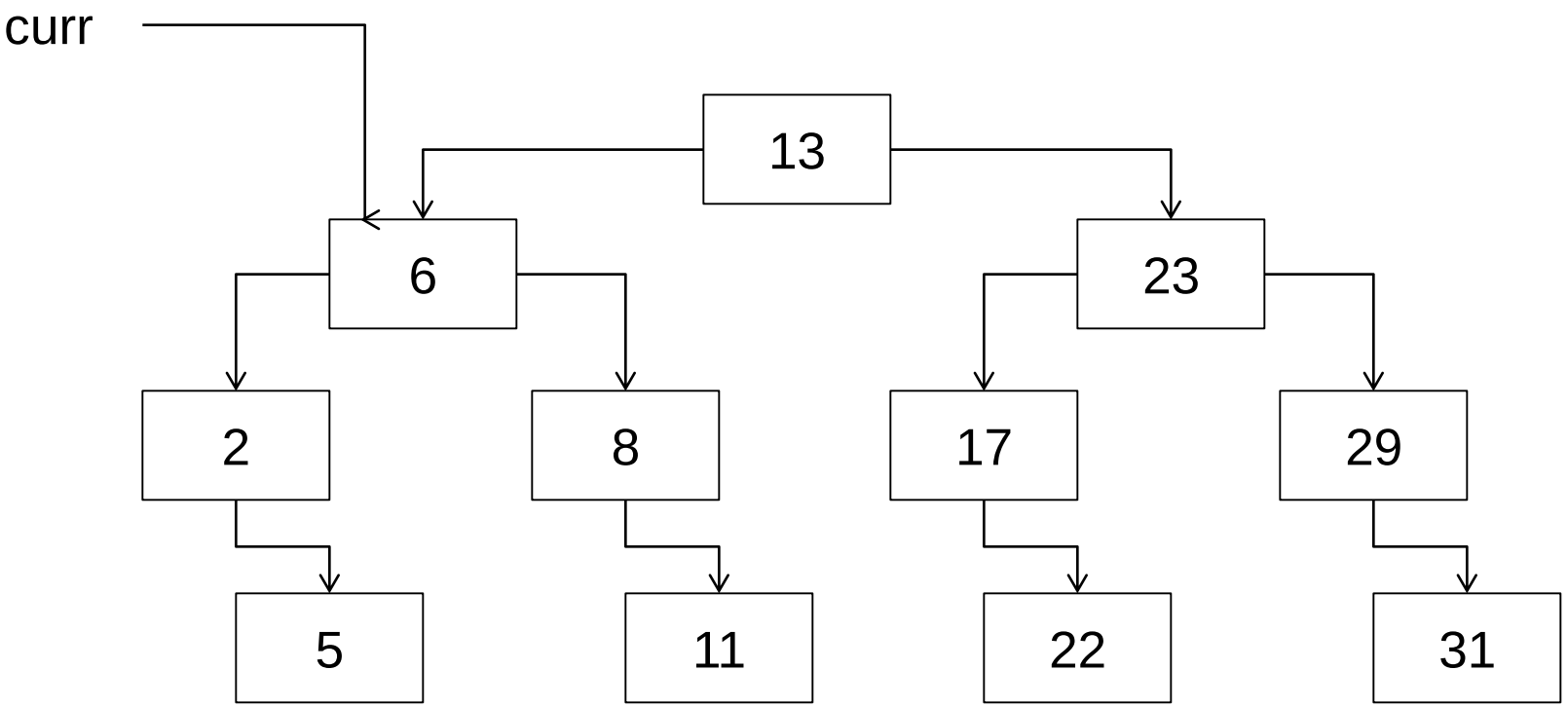
Trees and Contains

- Search for 5
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



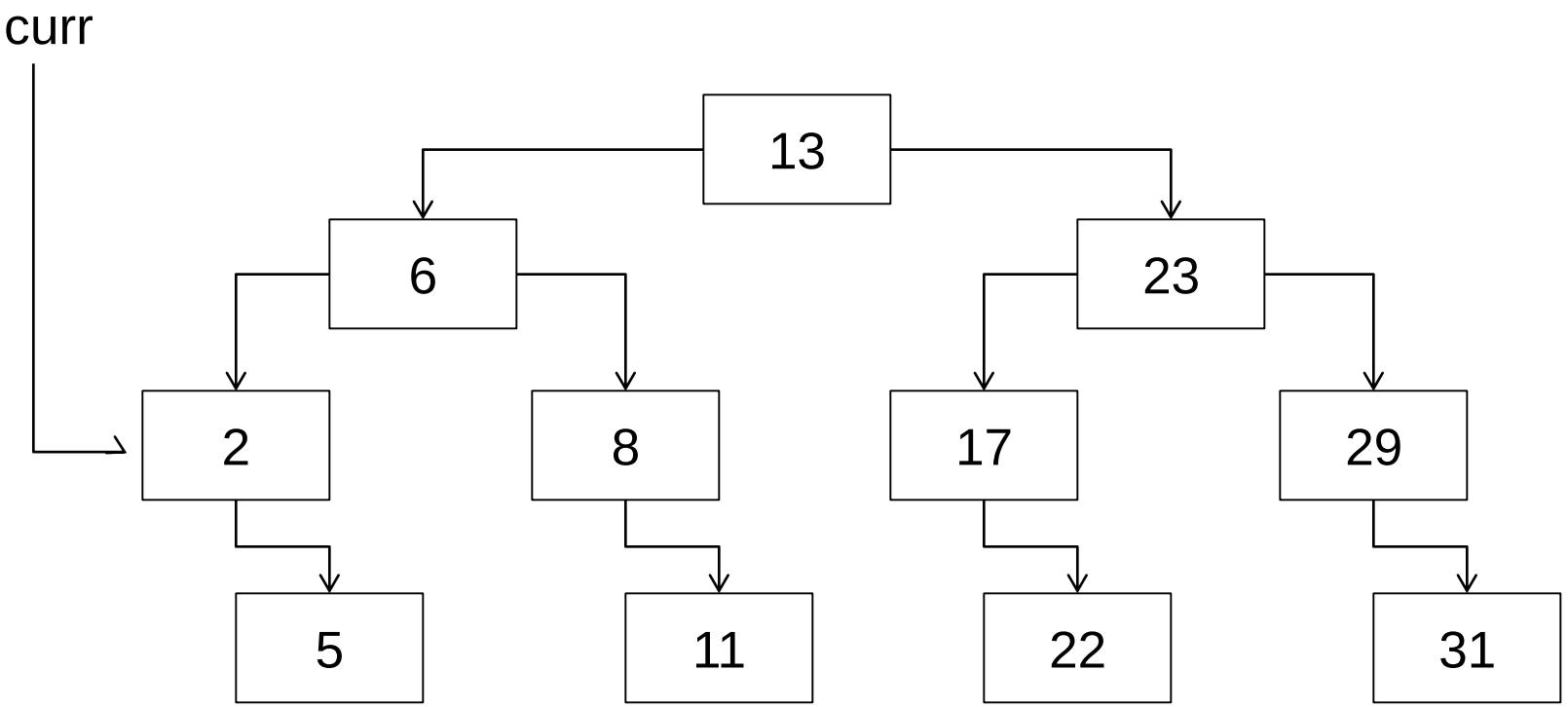
Trees and Contains

- Search for 5
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



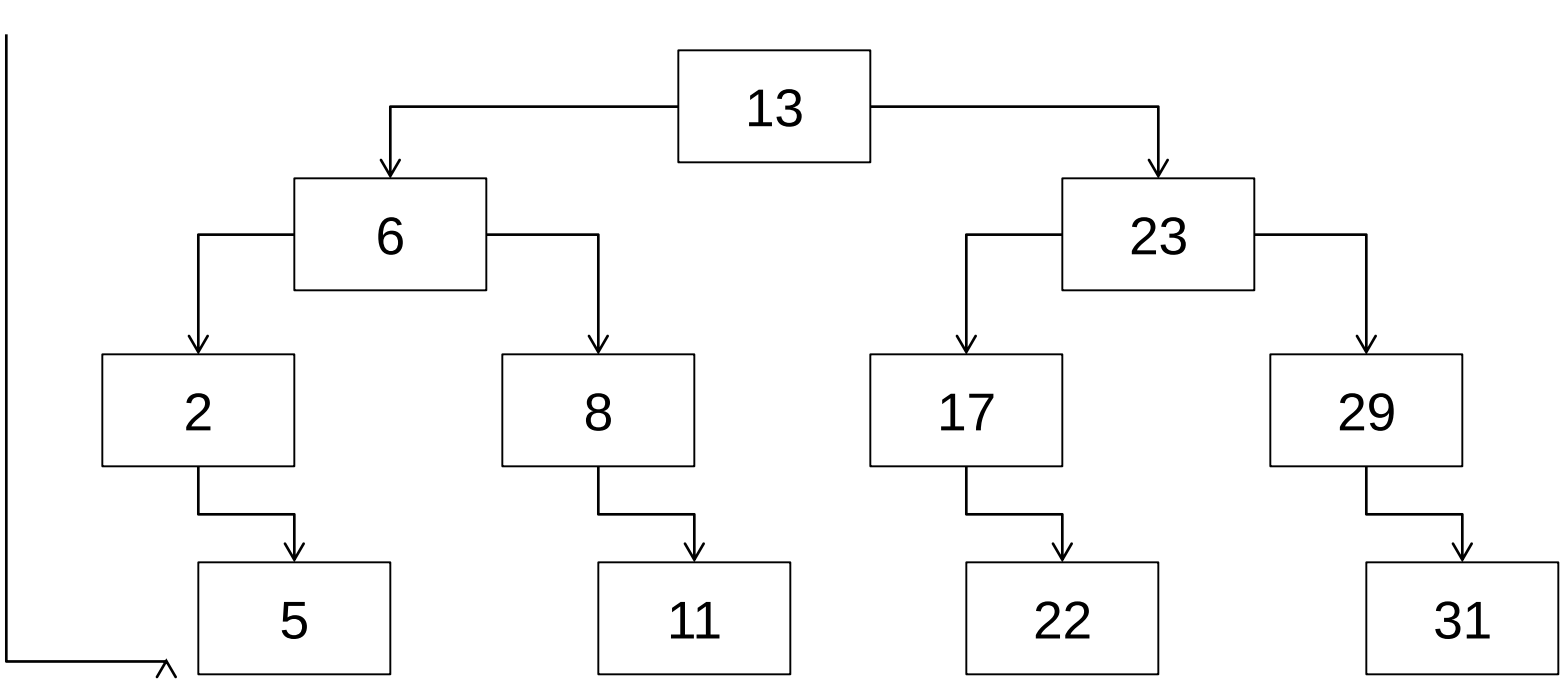
Trees and Contains

- Search for 5
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



Trees and Contains

- Search for 5
- Start at root:
  - If root is too big, go left (entire right subtree is too big)
  - If root is too small, go right (entire left subtree is too small)



Insertion

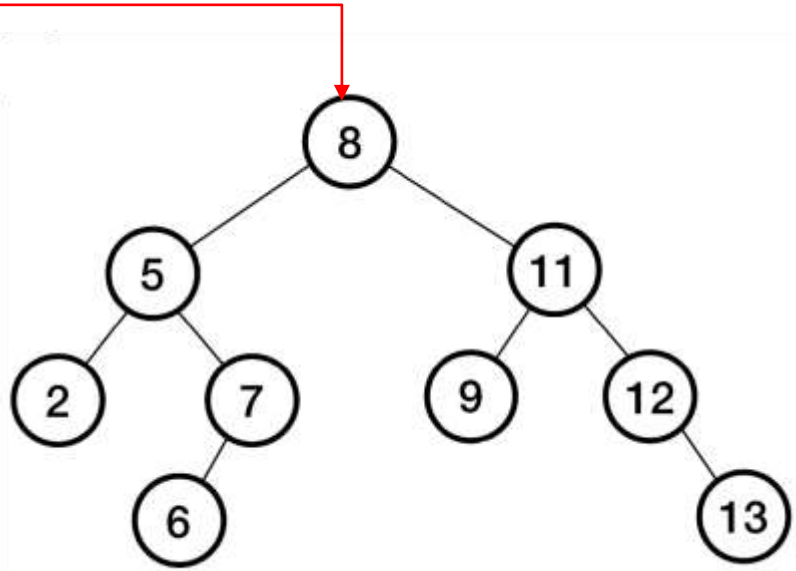
```
void Binary_Search_Tree<T>::insertion(node<T> *&p,T x)    //must add & (make it by address)
{
        if (p == NULL)
        {
                p = new node<T>;
                p->left = NULL  ;
                p->right = NULL ;
                p->data = x ;
        }
        else
        {
                if( x < p->data)
                        insertion ( p->left , x);
                else
                        insertion ( p->right , x);
        }
}
```

Insertion

For the following tree insert the value 10

Start at root

- Compare value with current node:
  - If value < node
    Go Left
  - If value > node
    Go Right



10 > 8

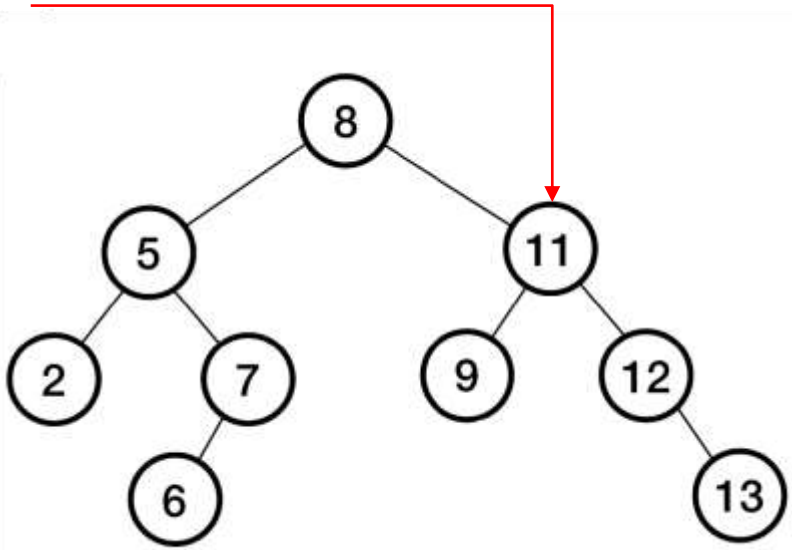
Go Right →

Insertion

For the following tree insert the value 10

Start at root

- Compare value with current node:
  - If value < node
    Go Left
  - If value > node
    Go Right



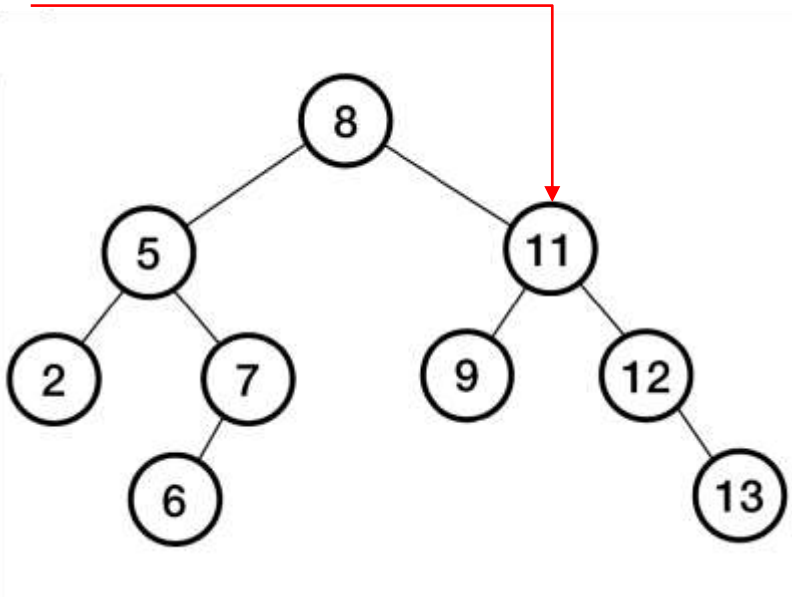
10 > 8

Go Right →

Insertion

For the following tree insert the value 10

- Compare value with current node:
  - If value < node
    Go Left
  - If value > node
    Go Right



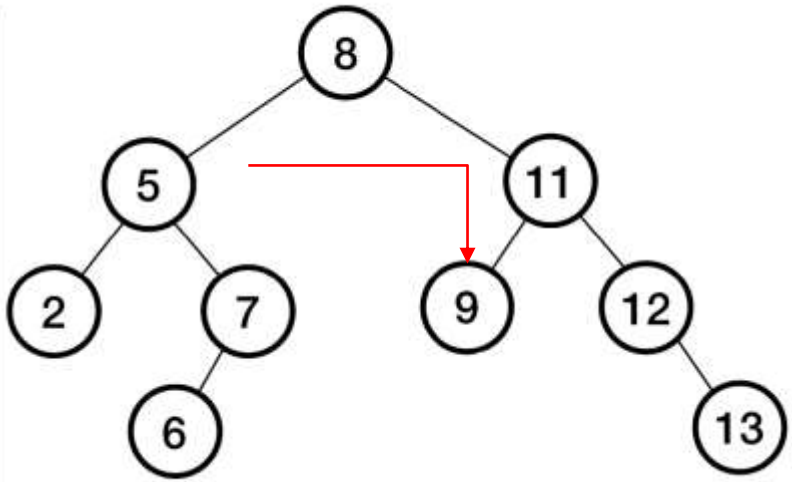
10 < 11

Go Left ←

Insertion

For the following tree insert the value 10

- Compare value with current node:
  - If value < node
    Go Left
  - If value > node
    Go Right



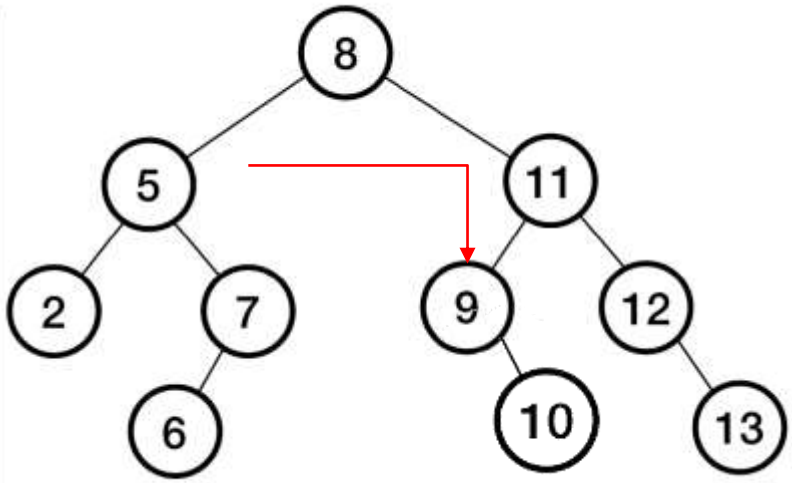
10 < 11

Go Left ←

Insertion

For the following tree insert the value 10

- Compare value with current node:
  - If value < node
    Go Left
  - If value > node
    Go Right



10 > 9
Go Right →

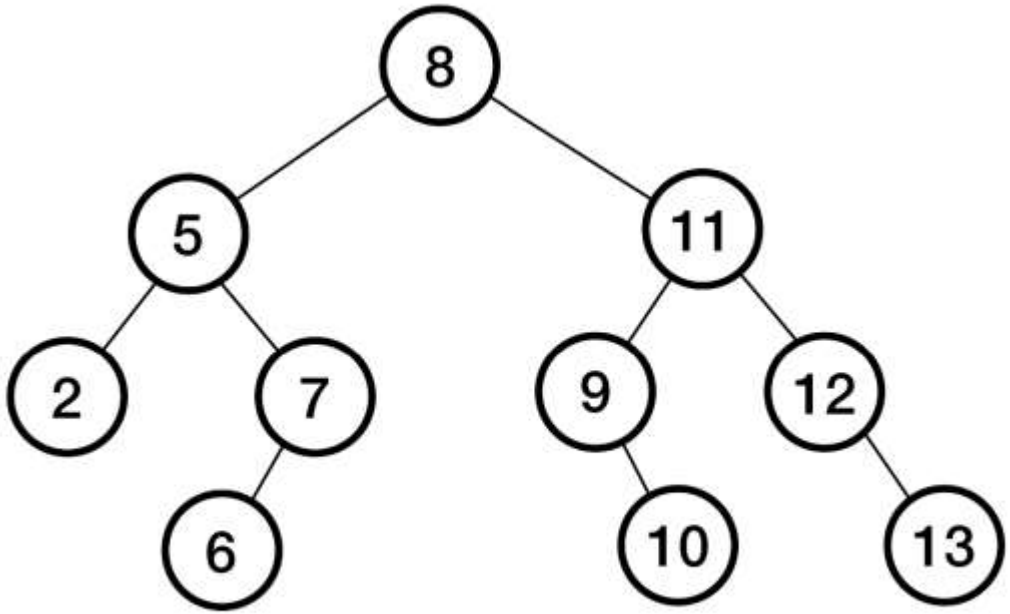
No node exist here,
then insert value here

Insertion

You have an empty binary tree, insert the following values in order. Then, show the tree formed.

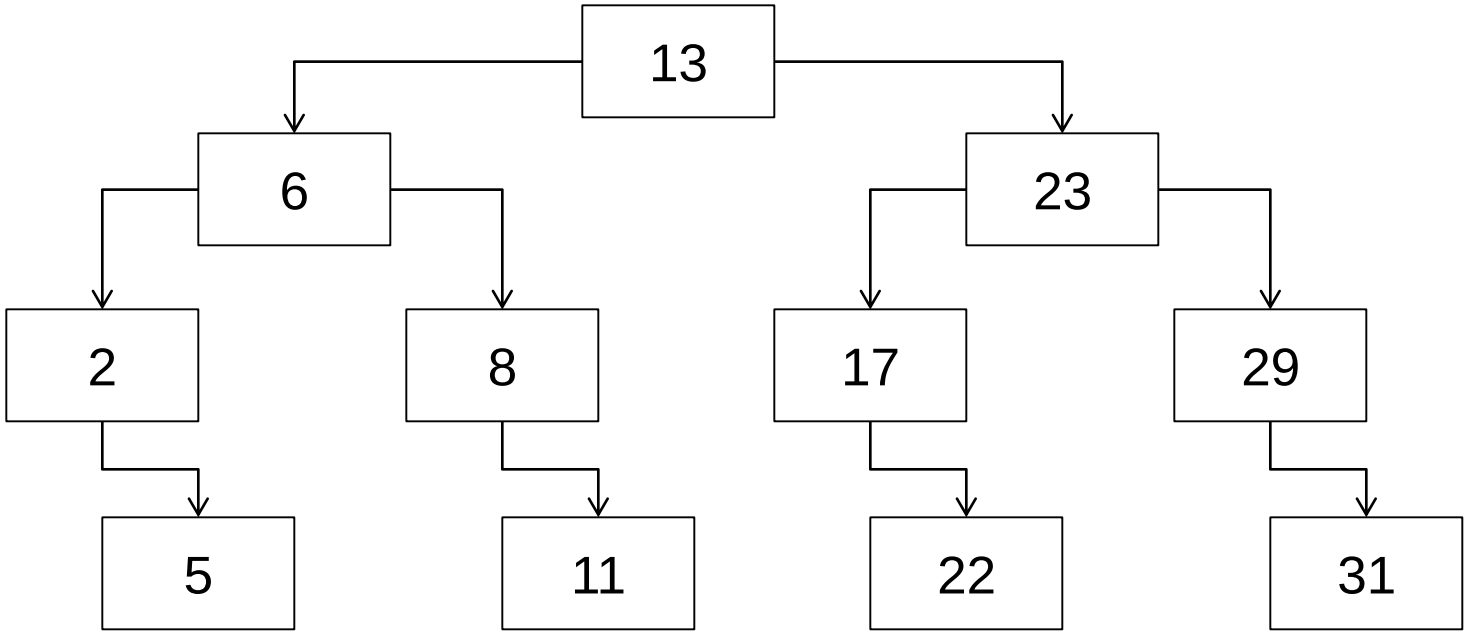
values = {8,5,11,2,7,9,12,6,10,13};

The tree formed:



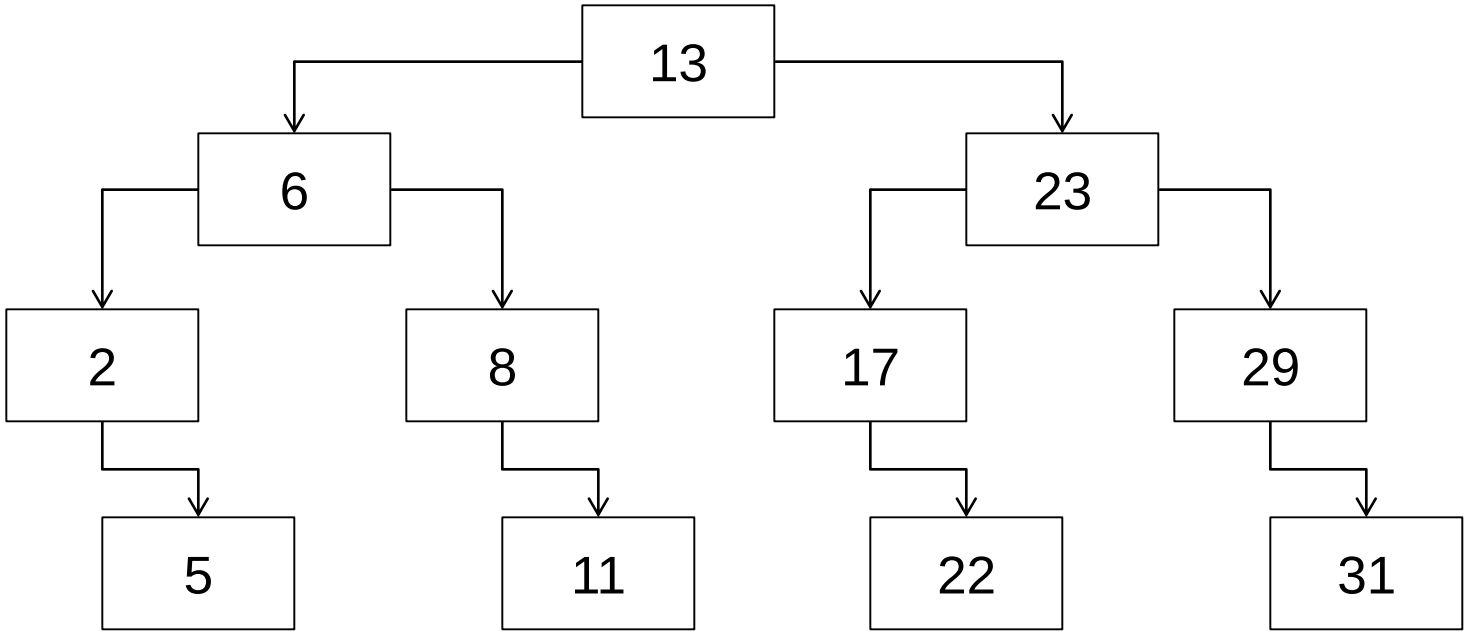
Printing Trees

- We need to be able to print our Set
- How would we print a tree?



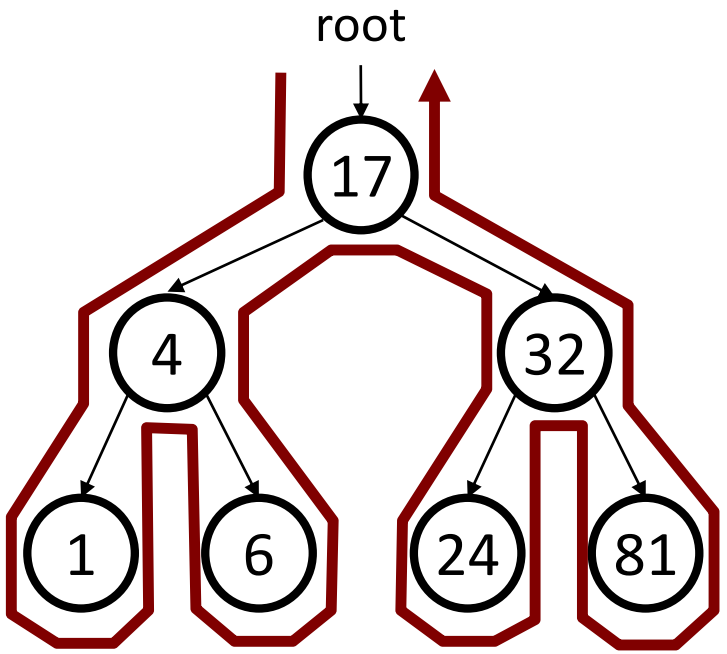
Printing Trees

- How would we print a tree?
  - Need to recurse both left and right
  - **Traverse the tree!**
    - Most tree problems involve traversing the tree



Traversal trick

- To quickly generate a traversal:
  - Trace a path counterclockwise.
  - As you pass a node on the proper side, process it.
    - pre-order: left side
    - in-order: bottom
    - post-order: right side
- What kind of traversal does a for-each loop in a Set do?



- pre-order: 17 4 1 6 32 24 81
- in-order: 1 4 6 17 24 32 81
- post-order: 1 6 4 24 81 32 17

Pre-order, In-order and Post-order

Let's see the code for simple pre-, in-, and post-order traversals!

```
void preorder(Node *p)
{
  if (p == nullptr)
    return;
  cout << p->value;
  preorder(p->left);
  preorder(p->right);
}
```

```
void inorder(Node *p)
{
  if (p == nullptr)
    return;
  inorder(p->left);
  cout << p->value;
  inorder(p->right);
}
```

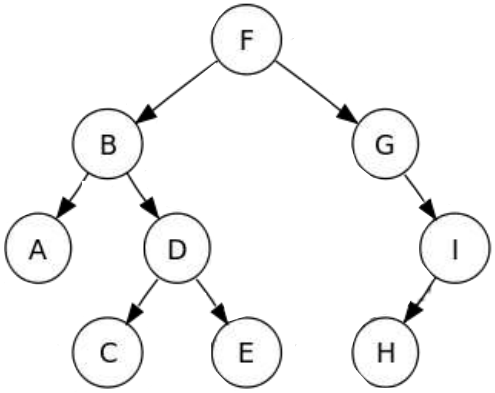
```
void postorder(Node *p)
{
  if (p == nullptr)
    return;
  postorder(p->left);
  postorder(p->right);
  cout << p->value;
}
```

Notice that the only difference is placement of the processing step!

But that small change results in major differences in the order the nodes are processed!

Ok, let's trace through our pre-order traversal!

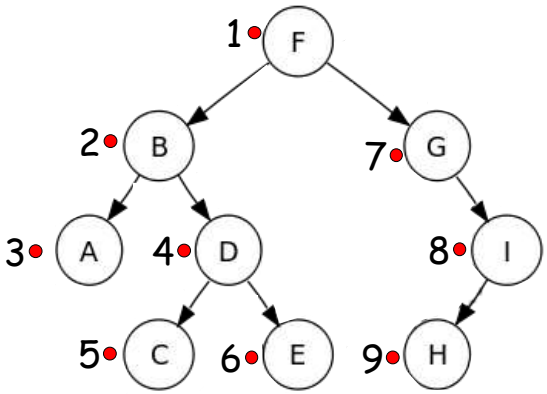
An Easy Way to Remember the Order of Pre/In/Post Traversals



Starting just above-left of the root node, draw a loop counter-clockwise around all of the nodes.

Ok, got that?

Pre-order Traversal: Dot on the LEFT



Pre-order:
F B A D C E G I H

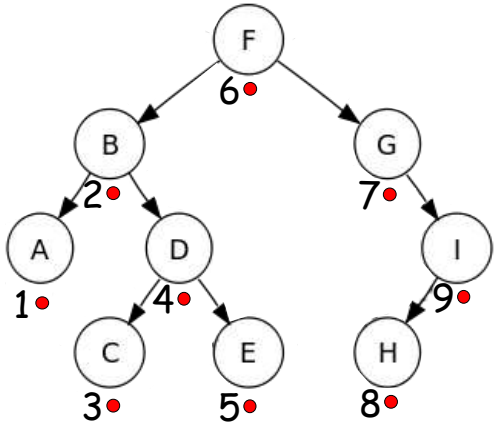
To determine the order of nodes visited in a pre-order traversal…

Draw a dot next to each node as you pass by its left side.

The order you draw the dots is the order of the pre-order traversal!

```
void preorder(Node *p)
{
  if (p == nullptr)
    return;
  cout << p->value;
  preorder(p->left);
  preorder(p->right);
}
```

In-order Traversal: Dot UNDER the node



In-order:
A B C D E F G H I

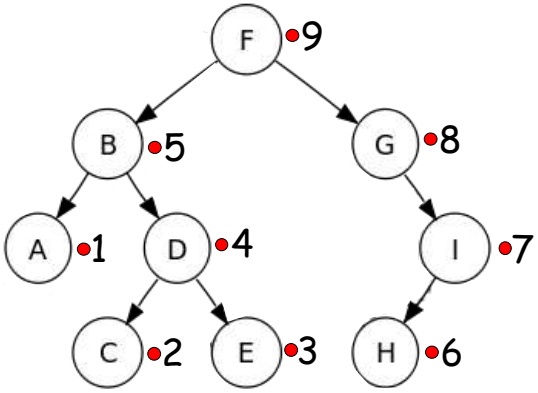
To determine the order of nodes visited in a in-order traversal…

Draw a dot next to each node as you pass by its under-side.

The order you draw the dots is the order of the in-order traversal!

```
void inorder(Node *p)
{
  if (p == nullptr)
    return;
  inorder(p->left);
  cout << p->value;
  inorder(p->right);
}
```

Post-order Traversal: Dot on the RIGHT



Post-order:
A C E D B H I G F

To determine the order of nodes visited in a post-order traversal…

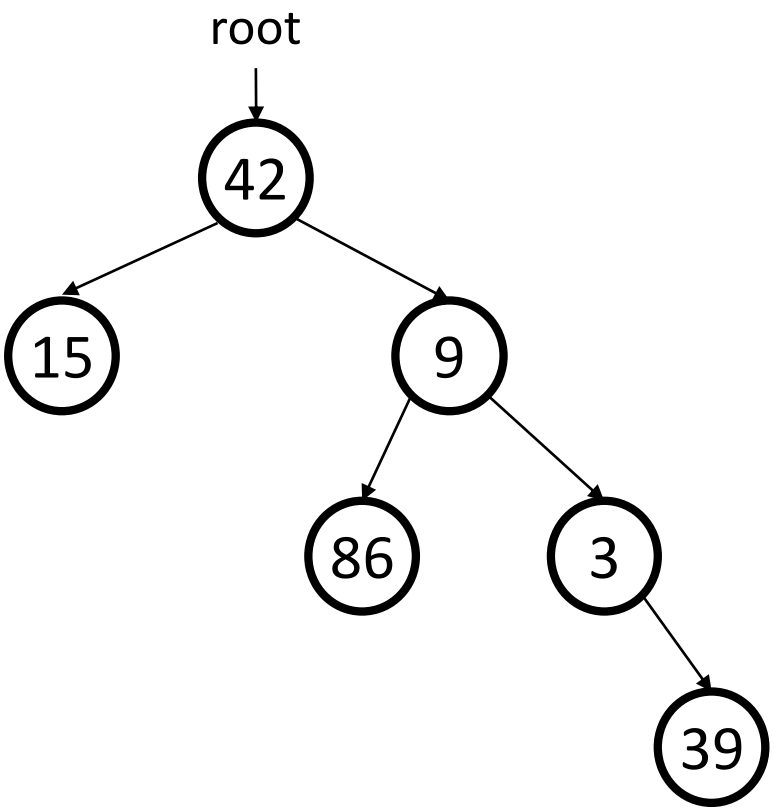
Draw a dot next to each node as you pass by its right side.

The order you draw the dots is the order of the post-order traversal!

```
void postorder(Node *p)
{
  if (p == nullptr)
    return;
  postorder(p->left);
  postorder(p->right);
  cout << p->value;
}
```

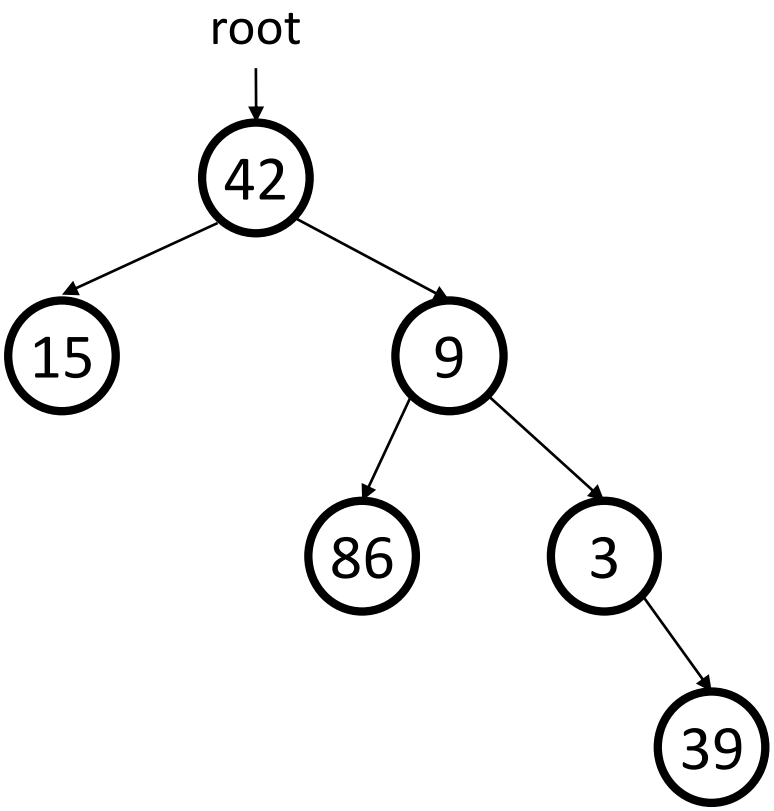
Traversal exercise

- Give pre-, in-, and post-order traversals for the following tree:



Traversal exercise

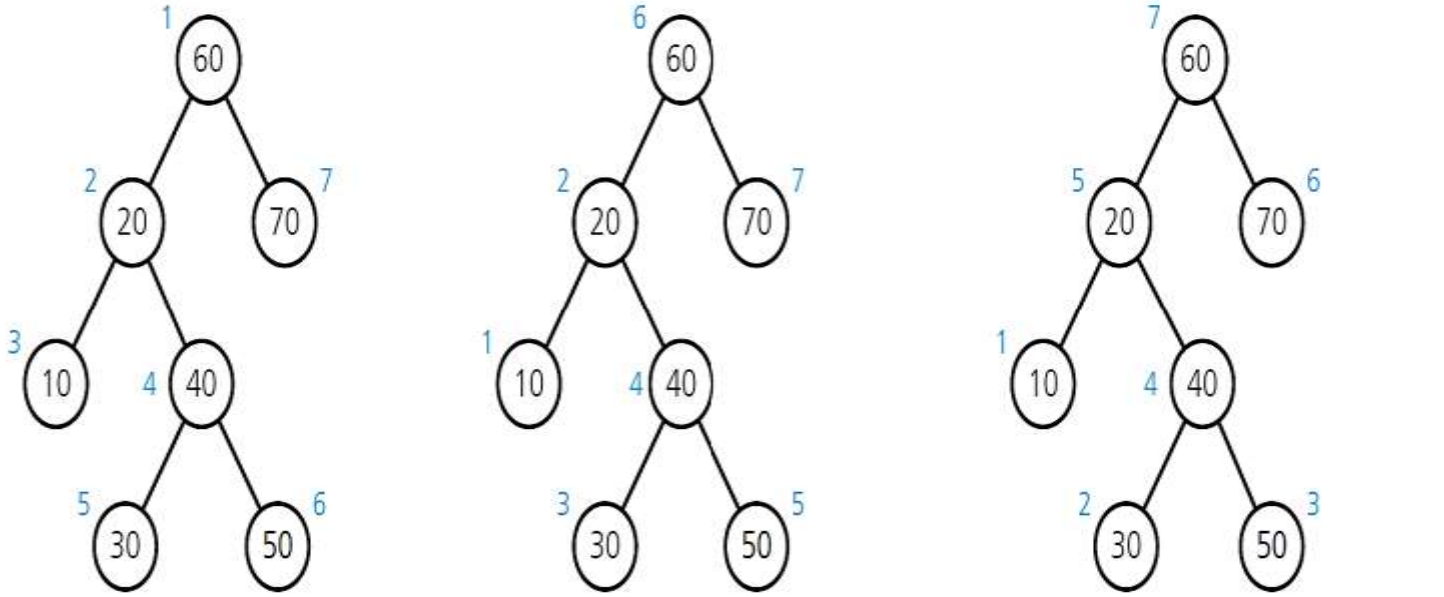
- Give pre-, in-, and post-order traversals for the following tree:



  - pre: 42 15 9 86 3 39
  - in: 15 42 86 9 3 39
  - post: 15 86 39 3 9 42

Traversal exercise

- Give pre-, in-, and post-order traversals for the following tree:



(a) Preorder: 60, 20, 10, 40, 30, 50, 70 (b) Inorder: 10, 20, 30, 40, 50, 60, 70 (c) Postorder: 10, 30, 50, 40, 20, 70, 60

(Numbers beside nodes indicate traversal order.)

Binary Tree Traversals

Ok, now we know how the recursive tree-traversals work.

Now let's see the level-order traversal.

The level-order traversal doesn't use recursion!

Instead it uses a queue, and processes the nodes of our tree much like we did with our maze search!

(in breadth-first order)

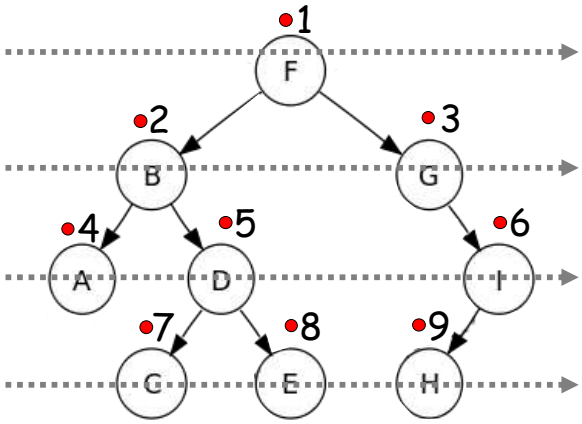
Pre-order Traversal	In-order Traversal	Post-order Traversal

These traversals all use recursion, and are nearly identical algorithms.

Level-order Traversal

L.O.T. uses a breadth-first-search to visit nodes.

Level-order Traversal: Level-by-level



Level-order:
F B G A D I C E H

To determine the order of nodes visited in a level-order traversal…

Start at the top node and draw a horizontal line left-to-right through all nodes on that row.

Repeat for all remaining rows.

The order you draw the lines is the order of the level-order traversal!

55